

OCR GCE A

COMPUTER SCIENCE

PROJECT

H446-03

Name : Eesha Gohil

Candidate Number: 8065

Harris Academy Greenwich: 10626

Title of Project: Chai Central: Restaurant Game

H446-03 – PROJECT CONTENTS

TABLE OF CONTENTS

Table of Contents	2
A. Analysis	7
Introduction	7
Stakeholders involved	7
Analysing similar games	7
Cooking Fever	8
Good Pizza, Great Pizza	9
Differences I've noticed between the two games that make them unique	10
Interview with the stakeholder	11
My features to the game	12
A few limitations	13
Hardware and Software Requirements	14
Computational methods	14
Thinking abstractly	14
Thinking Ahead	15
Thinking Procedurally	15
Thinking Logically	17
Thinking Concurrently	17
Specific and measurable success criteria	18
B. Design	19
Systems diagram	19
Explanation of my game objects	20
Breaking it down systematically	21
My Graphic development – main screen	21

Variables	23
Explaining the usability features	25
During the game	25
The main and menu screen	27
Starting video	28
Settings screen	29
Endgame	29
Extra parts of the game for usability.....	30
The testing of code.....	31
How will my game get increasingly difficult?	31
Test Data	32
Pseudocode for my game	33
The video module:	33
Pause screen:.....	34
recipe book screen:.....	35
Procedure startup:.....	35
Procedure for user clicks on objects on screen:	36
Function for tea names and recipes:.....	36
Function for the actual game:	37
Everything in the while loop for the entire game:	38
for events for whole game	40
Test data for development	41
Post test data	42
Class diagrams	43
Further data	45
C. Developing the coded solution (“The development story”)	45
The video 22/11/22	45

Client opinions.....	46
Making Buttons.....	47
Slight acknowledgements 4/12/22	49
All basic variables 19/01/23	51
Confirm button 23/10/23.....	51
Client opinions.....	52
Making screens	52
Making Title screen 23/01/23	52
Making Levels screen 26/01/23.....	54
Making Tutorial level 06/12/23	55
Making the settings screen 07/12/23 1:27pm	56
Client opinions.....	56
Attempting game state manager 11/07/23.....	57
Classes	58
Creating a sprite class 10/09/23	58
Creating a timer class 20/09/23	59
Making a timer 22/09/23	60
Client opinions.....	63
Creating Hit boxes 15/08/23.....	64
Hit boxes: Ongoing error	65
Creating the concatenation of user inputs 23/10/23.....	66
Client opinions.....	67
Sound effects 16/08/23.....	67
Client opinions.....	68
Creating a tuple (Aim of the game)	68
Attempt 1 23/10/23.....	68
Attempt 2 10/11/23.....	71

Sub screens.....	72
The pause screen 09/09/23.....	72
Client opinions.....	73
The recipe screen 15/11/23	73
Client opinions.....	75
Creating procedures for the main game.....	75
Startup Procedure 30/10/23	75
Game Procedure 07/11/23.....	76
Client opinions.....	78
“Esc” key error 01/12/23.....	79
Issue with using unnecessary global variables 04/12/23.	79
The end game (game state 10) 04/12/23	79
Client opinions.....	80
Issue with the scoreboard 06/12/23	80
D. Evaluation	81
Final thoughts – With client.....	81
Criteria that has been successful	81
Maintenance issues/ limitations.....	82
Improvements on limitations	82
Unmet or partially met criteria.....	83
Solving unmet / partially met criteria	83
Main Success Criteria	83
Success criteria detailed.....	85
Criteria of usability	87
The menu screen success criteria	87
The video at success criteria.....	87
Level screen at success criteria.....	88

Candidate Name: Eesha Gohil

Candidate Number: 8065

Main Game at success criteria.....	88
Recipe / pause screen	88
Robustness	89
Let's Play (Video) w/ timestamps.....	89
Project Appendixes.....	91
Code Listings	91
Disclaimer	104

A. ANALYSIS

INTRODUCTION

This single-player game consists of being time efficient, accurate and having good memory. It is similar to the app 'Cooking Fever' or 'Good Pizza, Great Pizza' where there are a variety of different teas to make accurately and give it to the customer. The aim of the game is to keep customers happy, sort of a microcosm for reality. Making the teas they ask for must also be done correctly such as pressing on the correct items in order as its required recipe. This is an extended game and other in game tasks will be difficult to enhance the difficulty of the game. This is also to ensure the game does not get boring since it is too easy.

STAKEHOLDERS INVOLVED

My target audience is more on the adolescence side since it includes animated characters and many eye-catching colours. However, the game may be too complex for children under the age of twelve as it requires you to create dishes in the correct order and requires players to work under pressure.

From the beginning, I wanted this game to give gamers an estimate of working in retail. Children under the age of twelve are not quite meant to work under pressure and their attention span is low if they are unable to complete the game, but the game should be fun and give you at least a slight bit of adrenaline. Therefore, the age rating is relatively high for this reason. The game is being created on PC however I aim to recreate this game on mobile so it can be played outside the house such as on a road trip in the car or just before school starts in the canteen.

I have also decided to make this game a little lengthy. This is to prevent the players replaying the same level with the same complexity. However, you can replay the level as the customers' requirements (the teas required to make) are randomised every time you click start. There are many levels to complete getting progressively more complex the more you play the game. In order to enhance the attention of the players, as you continue playing the game, I want to add little decorations they can personalise the look of the background and be able to unlock new dishes, drinks and more.

ANALYSING SIMILAR GAMES

Two games I decided to look into were "Cooking Fever" and "Good Pizza, Great Pizza". Since I have both these games downloaded on mobile, I know how to play both, and each are different in their own way. Here is a screenshot of Cooking Fever generally. Here, they have more than one customer on the screen at the same time. The player must reach a certain amount of money per level as seen at the top of the screen. The top of the screen also indicates how many customers there are left to serve and an experience ("XP") bar. Sound also plays a large part in the game since whenever the drink machine or oven has finished its job, it makes a "ding" sound.

COOKING FEVER

The initial release date of this app was on the 19th of August in 2014 and it is available on android, IOS and windows. Although there are many cooking related apps and games out there, this app was the one that made my childhood and I have it downloaded and still enjoy to this day.



Cooking fever doesn't really have a title screen however it has this layout which allows you to play in different settings with different levels, dishes, cultures, and customers. And at the top, it also contains your lifetime XP, cash and diamonds, another currency they have in the game. At the bottom, the producers have added a championship allowing the user to play against many other players across the world. Perhaps if I had more time for this project, I could've added this feature too.

Background is not changeable since the game is made for the user to play in different environments.

Tells the player how many customers left to serve until the end of the level.

XP bar at the top indicating your progress.

Many dishes can be created as the level increases in complexity.



Customers' patience level while waiting for food. Bar decreases every second. Half time goes from green to amber. Below 25% left, goes from amber to red. Once bar is gone, customer gets angry and leaves.

GOOD PIZZA, GREAT PIZZA

My game will be more comparable to “Good Pizza, Great Pizza” than “Cooking Fever” will be. The way this game works intrigues me and therefore, I have decided to add more ideas from this game. It involves a whole story line including you owning your first pizzeria; paying rent; taking care of deliveries; giving cash or free food to the homeless and even cash being deducted from what you earn in the day based on how many ingredients you used. Either way, the level is always completed even if you make no profit in the day. It just means that you may need to replay the level(s) to gain more money to buy ingredients and then you can proceed. This makes the game way more realistic than Cooking Fever. Its release date was on the 2nd of December 2014 and the developers are a group called “TAPBLAZE”.

Start button, also showing my real progress of the game.



This is the title screen of the game. It presents your progress in terms of in game days that you have completed. As you can see, the creators have implemented different chapters. This is my progress at the moment in the game. At the top right, there is a settings button and a plant pot looking button which allows you see the ingredients you have or can buy, and various accessories for your shop.

Customer's happiness level. The face goes unhappy below 50% then angry below 30%.

Odd things customers ask for making the game a little complex.

Amount of money you have.



Your progress and how long you have left of the current day.

Many customizable items that allow the shop to be personalized. Tablecloth, items on the desk, wall and floor patterns and colours, paintings such as that Mona Lisa etc.



Here is another screenshot of the game. When you go off the make a customer's order, the game automatically sends you to this cooking section which requires you to complete each step accurately, put it in the oven and cut it correctly before it is served. The screen switches back to the customer and the shop (the previous screenshot) once the correct number of pizzas has been put in the boxes.

DIFFERENCES I'VE NOTICED BETWEEN THE TWO GAMES THAT MAKE THEM UNIQUE

The difference between this game and "Cooking Fever" is that your shop is open from 12:00pm to 9pm so customers arrive only when you have completed someone else's order, and the time continues even while you are making the pizza(s). This means customers arrive one at a time, which I will be applying in order to make this achievable. Customers' happiness level is portrayed at the top of the screen as a percentage going down every second, which I will also be using. Correspondingly, the game "Good Pizza, Great Pizza" also has a customisable background but in more depth creating more personalisation for the user. With this game since it is only a pizzeria, you do not unlock dishes, you unlock different ingredients to put on the pizza. This includes unusual toppings such as aubergine slices and pesto sauce. Unlike Cooking Fever, this game also contains very regular updates for special occasions such as Christmas, St. Patrick's Day and Easter meaning limited edition items for the user to buy and collect.

INTERVIEW WITH THE STAKEHOLDER

Questions	Answers
What colours would you like to see within the game?	Green, Orange, White
How many items should be the max to create from one customer?	Four
What foods would you like to see added from the Indian culture?	Paneer, butter chicken, dosa, pani puri, samosa, bhajia, poppadom, chutney
Are there any other related games that you would like me to look into and add features from? If so, what are they?	Cake baking game, cooking mama, papa's pizzeria.
Would you rather the game contains many levels or just easy medium and hard?	Many levels
What sort of personalisation techniques do you want to see added into the game?	Customisation of music Random library
Which platform do you think is more suited for this game? And which platforms would you like to see it on?	Mobile Wants to see it on Xbox
How much time should be needed per customer/ order?	30 seconds

Here is the interview with my brother as the stakeholder, aged fourteen. His answers were a little biased since he chose all his favourite dishes. So, to gain a little more information, I asked others in the family for a variety of their favourites to include in the game. My brother's suggestion for customisation of music was something I had genuinely never really thought about so the survey was worth the time. Even though placing my game on Xbox would be a promising idea, I just do not think the time given is enough to make that achievable, but later, I may consider this idea and perhaps other consoles such as Nintendo switch depending on the overall development of the game. Most of the times set for the actual game itself is well planned out from my stakeholder and is something I agree with, so I will be taking this idea further. Colours from the Indian flag is a particularly innovative idea since it matches perfectly with the

overall them of the game. And lastly, both me and my brother had remembered playing a cake baking game when we sat down to talk about this, so I will be trying to find and take ideas from the game he had suggested me to research. Perhaps I can take those ideas further too.

A Disclaimer: This Q&A has been taken from the earlier stages of my ideas, so before the idea of the coffee shop that came about.

MY FEATURES TO THE GAME

Feature: A starting video

Justification: I want the user to have a real feel to the coffee shop and really make them feel like they are in the game. So, I will add this in.

Limitation: Causes the program to run slower since the video has a lot of memory.

Feature: Starting menu.

Justification: In order to create an interface for the user.

Feature: Levels menu

Justification: So, the user has an interface of a selection of levels to choose from.

Feature: Sound effects/ background music

Justification: This allows the game to be more realistic. The sound effects can also tell the user when an item has been pressed.

Limitation: If the user is playing the game in a loud environment or in a space where you want no sound, it can be hard to know when an item has been pressed. These environments defeat the purpose of having sound overall so ideally, my game should not totally rely on sound.

Feature: Recipe menu

Justification: This is so the user can familiarise themselves with the recipe of each tea in the game.

Limitation: Having a recipe book is vital for the game if you do not have the recipe memorised, but it can be quite long to read quickly and the abbreviations on the recipe screen might not make total sense at the beginning.

Feature: Tea Variety.

Justification: I want to add a variety of different teas. This is to embrace the reality of a real coffee shop and bring it close to home, India.

Limitation: Too many unusual types of tea can be extremely hard to remember for the user to make since each will all require a recipe (series of items to click).

Feature: One customer at a time

Justification: This is to prevent the screen becoming too hectic and confusing to the players.

Candidate Name: Eesha Gohil

Candidate Number: 8065

Limitation: The game could become too easy since I have not included any ambiguous sentences like in the game I researched "Good Pizza, Great Pizza".

Feature: Timer bar on the side of the displayed customer.

Justification: Like "Cooking fever", I will have this same idea since it is a good representation of the time remaining.

Limitation: This idea will take a while to test and add into the game.

A FEW LIMITATIONS

There are some limitations to features that I wanted to add but are not possible in the time given. An example of this is the championship section of the game where you are compared to players online against the world. Unlike Good Pizza, Great Pizza, I am unable to create regular updates for the game when special occasions occur, also due to time because I would have to make sure it is up to date every day.

I also wanted the user to be able to save their progress and/or synchronise their data from the cloud or a file. I feel as if this is a complex idea that I will simply not be able to complete and research for in the given time. However, it would be rather interesting to research anyway and I may add a little section of this to explain how this could work.

Another limitation I have is the fact that I am being too ambitious here. At the centre of my ideas to this game, I really wanted to go above here and create a moving screen whilst my buttons are being displayed. In addition to this, the pygame (movie.py) module is now unsupported which makes all my research a lot longer. Adding a video is also testing the memory file of my game too as it could make my game run slower.

My idea of adding settings to change the volume of the game will also be quite difficult. At this moment, I cannot find a solution that will be feasible in the game. Music and sounds in general will also be limited as I must consider that all this will be called inside a while loop and each sound within the sound file could iterate itself instead of the actual file. This could make the music sound buggy and robotic.

Another thing I wanted to add is having more than one sprite on screen at the same time. Adding this idea seemed to difficult and messy on screen. However, it would have been more fun for the user if I had decided to add this.

I am also going to struggle getting the game out to the community since there is no saving data function or scoreboard or anything that requires a cloud. Some functionalities that I have chosen to add may slow down the performance of my overall file because of how big it can get. And this is unfortunately not worthy of putting out to the community.

HARDWARE AND SOFTWARE REQUIREMENTS

Hardware	Justification
Keyboard	To do things like q for quit or c to continue
Mouse	To interact with the interface
Speaker	To hear the game to justify what has been pressed.
Touchscreen	To interact with the screen. I do recommend that you also have a keyboard since I have used keys in order to un-pause the game.
Software	Justification
Operating system (Android)	So, users cannot edit my game. Customer support is also readily available to me.
Language (python)	Easy language to use and my strongest language to code in.
PyGame library	Enabled so my game can present an interface to the user.
Random library	So, my program is able to randomly pick the order the customer wants.

COMPUTATIONAL METHODS**THINKING ABSTRACTLY**

In order to make my game less complex in terms of graphics, the use of abstraction will allow for me to use less storage for the game and will be less time consuming. For this, I thought combining the two games I analysed previously would help make the game a lot easier to understand whilst also including little unique features.

- Customers will still look like customers however they will not include as many pixels as 'Cooking Fever' for example. This is more achievable for me in the given time and will not impact the game whatsoever.

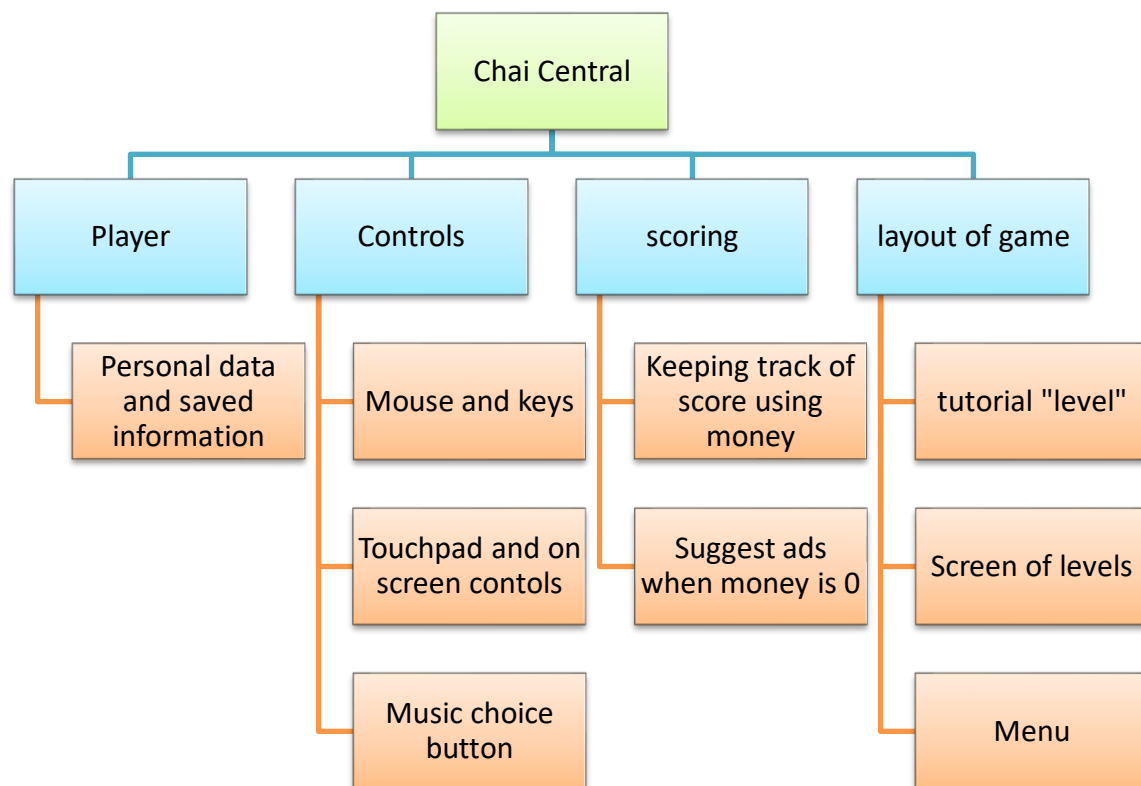
- The game will remain two dimensional like other games since having it three dimensional would be completely unnecessary. The game will run just as fine, and the player would be using the same controls and thinking in order to complete a level.
- A pause menu will be available at the top of the screen in order for players to see their lifetime progress, sound controls, volume, sensitivity, current level, change game mode and much more. This will be more necessary than providing a whole screen dedicated to this and will allow the user to track halfway through the game or save their progress.
- In 'Good Pizza, Great Pizza', the customers speak to you and it is your job to analyse what they say and create something based off it. For this game, just a picture of the tea within the speech bubble like 'Cooking Fever' is necessary. This creates the sense of virtuality since we do not speak in pictures.

THINKING AHEAD

Thinking ahead is going to allow me to maximise the code efficiency and minimise the errors made. To do this I will determine what the inputs, outputs and processes are that are needed for this project.

- A mouse to be able to click on distinct items within the game to produce outputs such as clicking ingredients needed to make a specific tea. For mobile, just tapping the screen allows output such as this.
- More outputs that I want is Audio. For example: a little music button as previously mentioned by my stakeholder. This allows the user to change the music of the game. There will be some built in music and also music that you can buy with in-game money.
- Also, visuals which are needed for every game.
- I also want data to be inputted into other processes such as when the name is entered by the user, it is said throughout the game such as during the tutorial when you first log on. Example: "Hi Pete, let's get started on the tutorial!"

THINKING PROCEDURALLY



My structure diagram consists of everything that I want to add into my game. Of course, there are even smaller details not added on the diagram as they would be seen as sub-sub tasks. Within my game there are four main tasks:

Player: I will require a screen showing all the user's information and progress of the game. For example: 62% completion of game.

-This screen will also include a username and email, if they would like music on or off, volume of game, sound effects on or off and more. All to the user's preference.

Controls: The controls will vary due to the different platforms I want this game to be on. On PC, classical mouse and keyboard will be considered. Mouse to click things, different keys to cook things, like hold s to stir.

-Mobile is slightly different as there is a touchscreen. Joystick to stir, swipe up and down to chop etcetera.

-A music button with customisation will also be available to the user.

Scoring: Scoring is also a little different to normal scoring in a game. It is completely normal to be on one coin for example. However, the game will notify you when you have run out of money and will suggest in-game purchases, restart a level or to quit.

Layout of game: There will be a few interactive screens such as settings, tutorial level and the actual game where the game can be played. The difference in all levels is the difficulty.

THINKING LOGICALLY

Here, I will talk about all my decision points within the program. This part is essential because without these points, the game will not run as intended causing a logic error. Just like any game screen starter, the first decision is the first screen.

- **Main menu:** On the screen there are three buttons, play, tutorial, and settings. The next screen is determined by the choice of the user. Many buttons like this will be displayed on top of all screens so the user is able to pause the game or go to settings during gameplay or choosing levels.
- **During gameplay:** When the user completes an order and gives it to the customer, if it is correct, the customer leaves and money is gained. If it is wrong, the customer will still leave however money is lost.
- **User choices:** When the user picks a level, there is a start or a go back. Then the level will begin based on the user's choice. The customisation for music feature is also a decision point for the computer. Based on the user's choice, the computer will continue iterating through music every time the user clicks the music button. Iterating will stop when the user stops clicking when the desired music choice has been found.

THINKING CONCURRENTLY

In this stage, I will be mentioning everything in my game that occurs simultaneously. Again, the majority of these things are incredibly significant for the game to be able to run smoothly.

- **Scoreboard:** For my game to run continuously, the money or score, must be increased or decreased while the user is still able to cook and complete orders (play the game).
- **Decision points:** I also want the game to keep running while some decision points occur. For example: When the customer's order is completed, the customer should be able to walk off the screen while the user is still able to interact with the game.

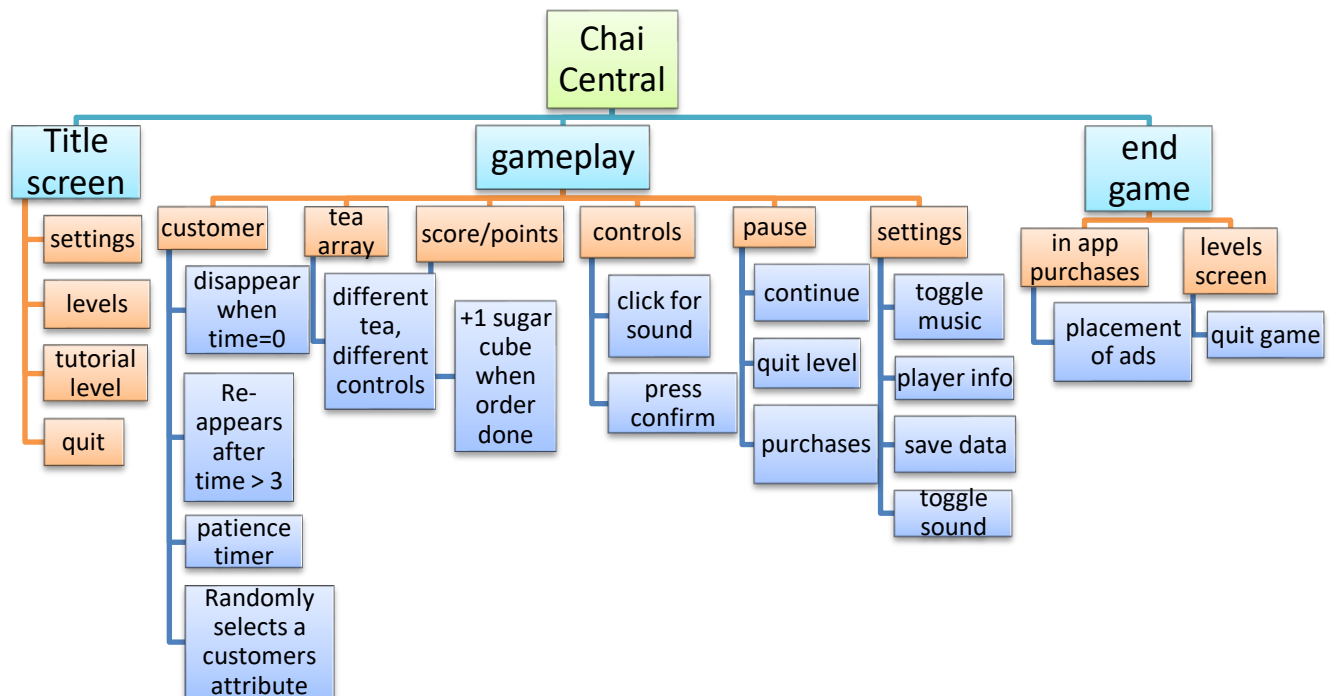
SPECIFIC AND MEASURABLE SUCCESS CRITERIA

Criteria	Justification
It should display a menu screen	The three different buttons I wanted, play, settings, and the tutorial. This prevents the game from being run immediately and the player can choose what they want to do first and gives a nice little interface before the game begins.
Customer's patience (timer)	Gives you a time for the order to be completed. Without this, the game is unachievable and non-realistic, there would be no point in the game overall.
Score (how much money you have as a whole)	Every order you complete, you will earn one sugar cube. Every order you fail, no sugar cubes are granted. Aim of the game is to get one sugar cube per round.
Item interaction on screen	Different items interact in diverse ways. So, I want the user to be able to do this. For example, mixing requires circular motions on the screen or dicing ingredients require a tapping motion on the screen.
Settings	So, the user can edit and modify the game to their liking, such as volume and sound effects.
Array of tea types	So, the user has to make and remember how to create a variety of teas which increases the difficulty of the game and makes the game less repetitive.
Stopping the iteration of the game (endgame screen)	When the game ends, a screen should load showing the score you got and the option to go back to the levels screen or quit the game. This is so the game is not on-going and updates the user of their score to see how well they did.
Random library	So, the tea types can be randomised so every level is not the same. Or even if you retry the level, the order from the customer to make is not the same.
Customers on screen	Should arrive one at a time and only leave the screen when their order has been completed.

Sound effects	Such as customer noises, people in the background like reality, like a real restaurant to enhance the game's features.
Background music	As suggested by my stakeholder and also to enhance the features of the game.

B. DESIGN

SYSTEMS DIAGRAM



EXPLANATION OF MY GAME OBJECTS

Title screen: This presents the first display of the game when you first switch it on.

Settings: This displays the technical parts of the game; it is further described in other boxes.

Levels: gives the user a place to choose what level they want.

Tutorial level: if you are new to the game, you might consider learning how to play the game. Step by step instructions are given on how to play.

Quit: Allows the user to quit the game.

Gameplay: Once you get past the title screen, the user is taken to the gameplay which is the interactive screen.

Customer array: The list of customers with their own appearances.

Action: This makes a random customer walk on screen, providing their order of teas.

2nd Action: The customer leaves and you do not gain points when their patience runs out.

Patience timer: A bar at the top representing the time the user has until the customer walks off.

Tea array: A list of all the teas that can be created and ordered.

Tea controls: Different teas require a different combination of interactions in order to be made.

Candidate Name: Eesha Gohil

Candidate Number: 8065

Score/ coins: Displays player's score count and lifetime sugar cubes. Includes an addition of +1 sugar cubes.

Controls: allows the player to interact with the screen.

Effects: players can click, hold, or tap a key to make teas.

Confirm button: use the button is used to serve to customers.

Pause: Allows the user to temporarily stop the game.

Continue: so, the player can continue the game after pausing it.

Quit level: so, the user can quit a level.

Purchases: the user can buy in game sugar cubes with money.

Toggle music: music can be switched on or off.

Player info: displays the users information needed in the game such as: recent purchases.

Save data: may not be included but allows user to save their data to the cloud.

Toggle sound: allows the user to control level of sound in the game and sound effects.

End game: This is presented to the user when the game ends.

In app purchases: Allows the player to buy in game things.

Ads: users can play an ad for some extra in game coins and random ads are placed every once in a while, to keep the game free.

Next level: Allows user to go the next level without going to level screen.

Retry level: Allows user to retry level.

Previous level: Allows user to go the last level played.

Quit game: Quits the game.

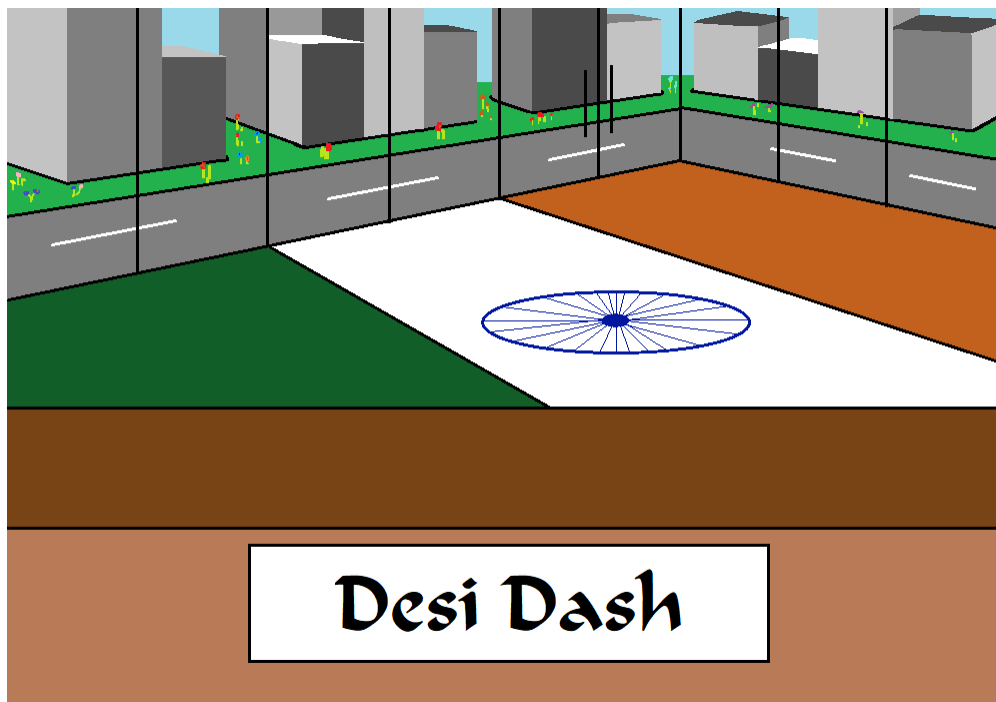
To clarify, this is what I expected my game to look like before the final product was made. It is also important to note that not all of these game objects were made successful in the final project.

BREAKING IT DOWN SYSTEMATICALLY

MY GRAPHIC DEVELOPMENT – MAIN SCREEN



This is the very first design of the title screen. I thought this would be necessary to show as my steps to my final design. Even though this approach out of my three models appealed most to the children, I wanted the game to look more sophisticated than this. Therefore, this did not make it to the final product.



Candidate Name: Eesha Gohil

Candidate Number: 8065

This is my second design for the menu screen. This was using Microsoft paint. After this I thought I should use paint 3D instead. Therefore, my final design uses paint 3D.



This is my final design, made in paint 3D. Here is also where I decided to change the name of the game. I had a few errors with my designs and lost all my creations. So, when it came to creating a contemporary design, it almost began to look like a coffee shop. Instead of changing my design yet again, I decided to keep it as it gave me lots of innovative ideas to go into. I also went from 2D to 3D since I thought it would give the user the best experience for this game. This design also allows me to re-use it for the title screen, main screen and even the video first displayed when the game is opened up.

VARIABLES

Variable	Data type	Justification
button	Boolean	Checks for user inputs and to go to different game states. This enables the user to interact with interface.
correction	string	Checks if the user has clicked the correct order of things for the recipe. Matches it with the one being asked for on screen.
Num	integer	This variable is significant since the scores collected from the user needs to be stored here in order to be outputted at the end screen so they can see their final score.
count	integer	This variable counts the iteration of each level. This means the rounds can be counted so the level does not go on forever.
Tea1 (recipe)	tuple	The recipe that has to match with the variable correction. This is a series of strings that concatenates together every time the user clicks an item.
Tea2 (name)	tuple	The name of the tea displayed on screen. This corresponds to the previous variable above this one. This prevents the actual answer being displayed on screen.
T0	float	This variable allows me to store a timer that starts as soon as the round starts.
T1	float	This variable allows me to store a time relative to PyGame.
dt	float	This is the difference between T0 and T1 so I can get a full timer which stores the length of change over between rounds which is 3 seconds.

T4, t2, dt2	float	This timer controls the time (patience) of the customer before the round ends.
Tea1	string	These variables simply separate the teas selected from the tuples and into a string.
Tea2	string	These variables simply separate the teas selected from the tuples and into a string.
Ingredients1	string	These variables separate the strings into strings from the tuple.
Ingredients2	string	These variables separate the strings into strings from the tuple.
Game_state	integer	Changes the game states

EXPLAINING THE USABILITY FEATURES

DURING THE GAME

During the game, objects do not just include the “customers” and the type of tea. They will also include the actual objects that the user will interact with. To do this, I will load the background and individually create “invisible rectangles” on the object on the image. And this will check for user clicks when hovered over the rectangles. When the user has clicked on the rectangle, depending on the object, (each rectangle is under a different variable to represent different objects) it will display a sound or phrase. Also, I want to be able to click on other object on the counter while coffee=filling for example.



1. During the tutorial level, if a rectangle has been pressed, it will display a phrase since it allows the user to get used to the game.

Another object I will be using is a class of customers. These people will only be displayed in the centre of the screen and cut off by the counter to make it more realistic. They will ask for a type of tea in a speech bubble and will stay there asking for it until their requirement has been met. So, if tea has been given, they will leave the screen. I want it so they leave the screen regardless of if the order is right or wrong. The only difference is that points will increase by one or stay the same.

Also, to save time and memory since the time I have is little, the look of all tea types will be the same.

THE MAIN AND MENU SCREEN

At the start of the game, it will constantly be checking for mouse clicks on the buttons. Each button has a different game state. Once “levels” has been pressed, another screen will have appeared with five more buttons with the same game state. In this case `game_state==5`. For memory, I want to try and get the `game_state` the same for each level but with a few tweaks of difficulty.



STARTING VIDEO

As soon as the game loads, a quick video is shown before the main menu loads. This is to give the feel that this is reality. My aim of this game is to make it as realistic as I can whilst using python.

I thought it would give a nice feel to the user to make them feel like they are in the game. The user also has the option to skip this video by simply pressing “s”. I wanted this feature since I can imagine how irritating it can get logging into the game after many times.



SETTINGS SCREEN

This screen will display a menu of all the diverse types of Indian tea you can make in the game. There will be a random automated way on how to create the tea in-game. Even if it is odd, linking back to the “good pizza great pizza” game I investigated. For example, washing a cup after you’ve “supposedly made the tea”. Also on the settings screen, there will be a display of controls and a way to change the volume of the background music or sound effects. This is because I want the user to have as much free customisation as possible.



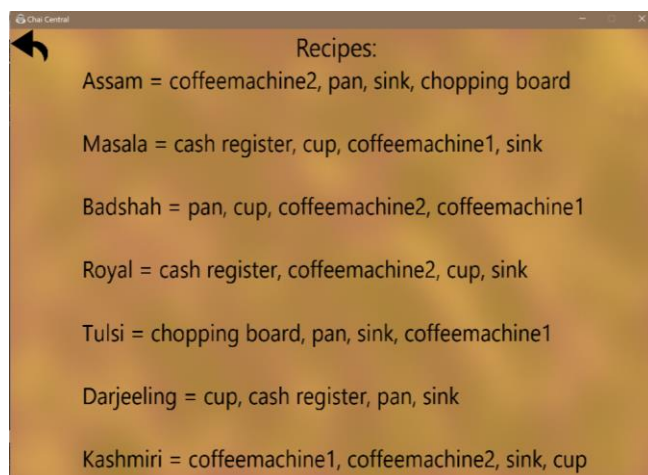
Pressing “r for recipes” is a trial part of the code that I wanted to do however I would have to make another game state within settings and will make the overall game a larger file and run slower. In addition to this, I no longer want a set recipe to make diverse types of tea, due to the explanation above the screenshot. With this method of making “customers” ask for randomly introduced strings of the variables, I can change the difficulty of the game easily. I can even increase difficulty based on how long the string is presented for on the string. This feature will be added to level 4 or 5 to show how the game can get difficult.

ENDGAME

Once you have completed the number of rounds the level provides, you will be greeted with this end game screen which tells you the maximum number of sugar cubes you could have received and how much you actually received. Then, when you want to go back to pick another level, simply press ‘e’ on the keyboard and it will take you back to the levels screen. If you want to quit the game, you can just click ‘x’ at the top of the screen.



EXTRA PARTS OF THE GAME FOR USABILITY



This is the recipe book for the game. The button to access this screen is only available to users for levels 1,2 and 3. This is because I wanted the game to get harder through levels 4 and 5. This way, through the first three levels, users are able to get to know the different types of recipes and can replay the levels as many times as they want.



This screen can be accessed anytime during the duration of the game by clicking the pause button at the top right of the screen. This is so gamers are able to pause the game and continue again if something has interrupted them. If they want to quit the game if the interruption is large, they can click 'q'. I want it so even if users quit the game, the progress can be saved. So, re-opening the game and clicking on that level, I would want it to continue from where the user left off with. For this, I would have to do more research in order to have this idea in my game.

THE TESTING OF CODE

In order to assess classes and objects individually, I use a separate file called test.py so I can evaluate what I am thinking about adding to my actual game. These also include prototypes of objects and more. The error I had earlier with the timer class was used in my test file so I could easily identify what the error was and why it was happening.

I also used procedures and functions to evaluate my game objects. This helped me to keep every other game object in its place and would not intervene with them.

HOW WILL MY GAME GET INCREASINGLY DIFFICULT?

Each level will get harder by the customer asking for a multiple number of actions for the user to complete. This will be represented by a code in which the user has to decode. For example, just as mentioned in my program, different objects have a simplified variable. So, coffee machine is cm1. This is so the game can get more complex, and the user must remember this code (after a certain amount of time before it disappears to complete the order.

TEST DATA**Menu screen**

What am I testing	Test type	predicted outcome
When mouse is outside of (x,y)=(1000,700)	erroneous	nothing
When mouse is over a button	boundary	nothing
When buttons are clicked	normal	Goes to different game state
When mouse.click on quit	normal	Game screen closes
User input = "esc"(intended to allow the user to go back to menu screen)	normal	Does nothing

Video at the start of the screen

What am I testing	Test data type	Predicted outcome
When game runs, video runs	normal	Video runs
User input = "s"	normal	Skips video to main menu

Levels screen

What am I testing	Test data type	Predicted outcome
User clicks a button	normal	Goes to game state five
User presses "esc"	normal	Goes back to main menu

Main game

What am I testing	Test data type	Predicted outcome
Sprite does not spawn	invalid	error

User clicks game object	normal	Displays a certain sound for a certain number of seconds
User clicks a non-game object	normal	nothing
Game objects are performed simultaneously	erroneous	error
Sprite spawns while user is completing "sprite's order"	Invalid	error
User clicks too many game objects at once	boundary	Can cause errors if program feels as if user is spamming mouse clicks.
customer count=5/7 on a level.	normal	Game ends, finish screen pop up loads
User presses confirm button (determines point increase)	Boolean	Point system goes up by one or points stays the same
Tracking user input (when clicking an object (part of the tea recipe)	string	Converts user input into a string that concatenates to an empty one.
Pressing recipe button	Boolean	Should change screen to recipe screen and pause the time.
Pressing pause button	Boolean	Changes to pause screen and should pause the game.
Checking keyboard presses in pause screen: "c" or "q"	Boolean	C to continue game or q to quit the whole game.
"esc" key to go back in screens	normal	Should go back to main menu when pressed at any time within the game.

PSEUDOCODE FOR MY GAME

THE VIDEO MODULE:

While video is running then:

 If user types "s" then:

 close the video.

 Output "exit has been requested."

 If video closes, then:

Output "error could not open video."

Else then:

Output "end of video, exciting."

PAUSE SCREEN:

procedure pause screen:

 play elevator music.

 paused = True

 while paused == True:

 for event in Pygame events:

 if event.type == pygame.QUIT:

 quit pygame.

 quit system.

 if event.type == pygame.KEYDOWN:

 if user clicks 'c' then

 paused = False

 stop elevator music.

 elif user clicks 'q':

 quit pygame.

 quit system.

 next event

 paint screen colour to white

 blit text to screen

 blit text to screen

 blit picture onto screen

update pygame

update the clock.

end procedure

RECIPE BOOK SCREEN:

procedure recipebook

 paused = True

 while paused == True then:

 for statement to ensure program can quit and can check for user input

 fill colour of screen

 blit the image onto the screen

 if return to game button clicked then:

 paused = False

 update Pygame

 update clock

PROCEDURE STARTUP:

Procedure startup:

 for statement to ensure program can quit and can check for user input

 fill screen colour

 blit main game image onto screen

 blit point image showcase onto screen.

 update Pygame

update clock

end procedure

PROCEDURE FOR USER CLICKS ON OBJECTS ON SCREEN:

Procedure main game:

Make correction variable global.

for statement to ensure program can quit and can check for user input

If user clicks on object, then:

Play sound corresponding to the object clicked.

Output a phrase corresponding to the object clicked.

Update Pygame

End procedure

FUNCTION FOR TEA NAMES AND RECIPES:

Function teastuff:

assam = "cm2"+"pan"+"sink"+"cb"

masala = "cr"+"cup"+"cm1"+"sink"

badshah = "pan"+"cup"+"cm2"+"cm1"

royal = "cr"+"cm2"+"cup"+"sink"

tulsi = "cb"+"pan"+"sink"+"cm1"

darjeeling = "cup"+"cr"+"pan "+"sink"

kashmiri = "cm1"+"cm2"+"sink"+"cup"

tea = tuple for defining each recipe as a name that can be displayed on screen

random tea from tuple

random tea from tuple

return the two recipes.

return the two tea names that will be displayed on screen.

end function

FUNCTION FOR THE ACTUAL GAME:

Procedure game:

Global all my variables

Start the game's timer.

If timer ≥ 3 then:

For statement to check user input

If user clicks "r":

Restart the recipe check (allows you to start again if mistake made)

Update customer class

Blit speech bubble onto screen

Start recipe timer.

If timer ≥ 50 then:

Restart the recipe timer.

Count = count +1

Update all the customer class.

Set recipe (correction variable) back to ""

Pick tea items randomly

Call the game function to check for user clicking game objects.

If recipe == correction recipe then:

Restart game timer

Point +1

Count + 1

Update new tea names.

Update customer class

Update Pygame

Else then:

Count +1

Restart game timer

Update tea names

Update customer class

Update Pygame

End procedure

EVERYTHING IN THE WHILE LOOP FOR THE ENTIRE GAME:

Game state 1:

Define game_state = 1

Define two timers.

If game_state ==1 then:

Fill screen with colour

Blit text onto the screen ("chai central")

If levels button clicked:

game_state = 2

If tutorial button clicked:

game_state = 3

if settings button clicked:

game_state = 4

if quit button clicked:

quit Pygame.

update pygame

Game state 2:

Fill screen colour

Blit text onto screen ("levels")

If level 1 clicked, then:

game_state = 5

if level 2 clicked then:

game_state = 6

if level 3 clicked then:

game_state = 7

if level 4 clicked then:

game_state = 8

if level 5 clicked then:

game_state = 9

update Pygame

Game state 3:

This is the tutorial level in which more research would have to be conducted before I can achieve this.

Game state 4:

This is the settings page which I am yet to finish.

Game states 5,6,7,8,9

Reset timers.

Count = 0

While game state == 5,6,7,8,9:

Call game function

If recipebook button pressed, then:

Go to recipe book page.

If count == 3:

Game_state = 10

Update Pygame

- Just a note that the recipe button code is only used in game states 5,6 and 7.

- Also, the count iteration (the number of rounds in the level) is three for game states 5,6 and 7. For game states 8 and 9, the count is five to make the last two levels harder.

Game state ten

fill screen colour

blit image to the screen

blit text onto screen (points earned at the end of the level)

update Pygame

for events in Pygame:

if user has pressed key, then:

if user has pressed 'e' then:

game_state = 2

if user has exited screen, then:

quit Pygame.

next events

FOR EVENTS FOR WHOLE GAME

For events in Pygame:

If user has pressed keys, then:

If user has pressed escape, then:

Game_state = 1

If user has exited screen, then:

Quit Pygame

Setting click input boxes to transparent.

Fill the input boxes white.

Blit input boxes onto main screen only

Update Pygame

Next events

TEST DATA FOR DEVELOPMENT

Menu screen

What am I assessing	Test type	predicted outcome
When mouse is outside of (x,y)= (1000,700)	erroneous	nothing
When mouse is over a button	boundary	nothing
When buttons are clicked	normal	Goes to different game state
When mouse.click on quit	normal	Game screen closes
User input = "esc"(intended to allow the user to go back to menu screen)	normal	Does nothing

Video at the start of the screen

What am I assessing	Test data type	Predicted outcome
When game runs, video runs	normal	Video runs
User input = "s"	normal	Skips video to main menu

Levels screen

What am I assessing	Test data type	Predicted outcome
User clicks a button	normal	Goes to game state five
User presses "esc"	normal	Goes back to main menu

Main game

What am I assessing	Test data type	Predicted outcome
Sprite does not spawn	invalid	error
User clicks game object	normal	Displays a certain sound for a certain number of seconds
User clicks a non-game object	normal	nothing
Game objects are performed simultaneously	erroneous	error
Sprite spawns while user is completing "sprite's order"	boundary	Rarely happens. true
User clicks too many game objects at once	boundary	error
customer count=0 on a level.	normal	Game ends, finish screen pop up loads

POST TEST DATA

Here is the data I will test for after I have completed my code. To carry out this data will require the presence of my client in order to try out the product so we can continue to make successive prototypes for the final product.

What Am I Assessing	Data type	Expected Outcome
User clicks too many game objects at once	boundary	error
User clicks game object	normal	Displays a certain sound for a certain number of seconds
User clicks a non-game object	normal	nothing
User presses "esc"	normal	Goes back to main menu
User input = "s"	normal	Skips video to main menu

I have combined the post data checks from every stage just because most of these things must be constantly maintained throughout my program.

I want to make sure that my game still runs even when a user queues up lots of tasks for my game to do, not for it to error and break the game. However, even though something may cause an error, it could spark an interesting idea that I could add to the game in the future. For example, sprites spawning simultaneously. I was originally aiming to have one sprite spawn at a time however this may be an interesting approach.

I have made my **normal** data, data that is meant to be entered, expected to happen normally as the game runs.

Next is **erroneous** data which is data that happens unexpectedly or the user inputs something unexpectedly.

Lastly, **boundary** data which I have recorded where user inputs are on border of what I want them to do. For example, I want the user to have the ability to click objects but not too many simultaneously. Or, while the game is running, it does not do exactly what I want it to do, sort of like a logic error.

CLASS DIAGRAMS

In this section, I will present my class diagrams for all my diverse needs within the game.

Customer class

Class Customer
Attributes: Self.x = integer Self.y = integer Self.talk = string Self.speech = string
Methods: Speech(self, message) Draw(self, surface)

In the customer class, I want every sprite to be able to provide a speech bubble where the user can read the order that they want. This creates usability for the user so it creates something to complete.

Including this, I want customers to randomly be chosen with a random gender and a random hair colour. This is to make the game more realistic.

Timer bar class

Class timer
Attributes: Self.x = integer Self.y = integer Self.w = integer Self.h = integer Self.timer = integer Self.max_timer = integer
Methods: Draw(self, surface) Colour(self, colour)

In the timer bar class, two rectangles are presented overlapping each other. Each bar will be drawn on the same place on the surface and each bar will be the same width and height. For each second that goes by, the second rectangle will start to decrease in length downwards to represent time running out.

This is just for the user to know how much time they have in order to complete what is on the screen. If the timer runs out, the game will continue to iterate through as though you had incorrectly completed the round.

Button class

Class Button
Attributes: Self.x = integer Self.y = integer Self.w = integer Self.h = integer Self.image = image Self.clicked = Boolean
Methods: Draw(self, surface)

Using another file (Button.py), I have put the button class inside and attached it as a module to my main game file (project_code.py). This way, all the button code is out of the way, and I can easily create new buttons with two or three lines of code. And this creates an interface for the user.

It has also allowed me to keep my game code well organised and much easier to read and understand.

I also decided not to make and draw my buttons using python but rather using my own images instead because it means I can create other shapes such as ovals, the ones I used in my main screen. The images I use match the theme of my game too, so everything looks in place.

FURTHER DATA

Here is some final data bits that I will be needing after I develop my game.

C. DEVELOPING THE CODED SOLUTION ("THE DEVELOPMENT STORY")

THE VIDEO 22/11/22

The first thing I wanted to do was create a video which displays under the buttons. I had a look at Minecraft's title screen when you first load in and I thought it would be cool to at least have a go.



However, when I did, the buttons would flash on the screen because it would blit to each individual frame running on the video. One of my peers suggested that I play the video then create a title screen which is then what it resulted in.

Test data:

What am I assessing	Test data type	Predicted outcome	Actual Outcome
When game runs, video runs	normal	Video runs	Video runs
User input = "s"	normal	Skips video to main menu	Skips the video to a screen that stays black momentarily before the actual program starts.

Video loads	Erroneous	Video is opened in the same window as the game	Opens a different window
-------------	-----------	--	--------------------------

The difficulty of this video is that the window is different to the window that my game is opened. And this can causes some errors for people who download my game since the windows are separate.

CLIENT OPINIONS

Successes:

- Video works as intended.
- User is able to skip video.
- Gives a realistic feel of the environment of the game.

Failures:

- Was not able to put video as part of the main screen.
- Video is opened in a different video.
- Video is worth a lot of CPU usage and memory for the game.
- Not really a requirement for the game itself.

```

Videomodule.py - C:\Users\eesha\OneDrive - Harris Federation\yr 13 computer science\All project materials\Videomodule.py (3.10.11)
File Edit Format Run Options Window Help
import cv2
import pygame

window_name = "Chai Central"
interframe_wait_ms = 30

cap = cv2.VideoCapture("Finalpiece.mp4")
if not cap.isOpened():
    print("Error: Could not open video.")
    exit()

cv2.namedWindow(window_name, cv2.WND_PROP_FULLSCREEN)
cv2.setWindowProperty(window_name, cv2.WND_PROP_FULLSCREEN, cv2.WINDOW_FULLSCREEN)

while (True):
    ret, frame = cap.read()
    if not ret:
        print("Reached end of video, exiting.")
        break

    cv2.imshow(window_name, frame)
    if cv2.waitKey(interframe_wait_ms) & 0x7F == ord("s"):
        print("Exit requested.")
        cv2.namedWindow(window_name, cv2.WINDOW_AUTOSIZE)
        break

cap.release()
cv2.destroyAllWindows()

```

Here is my code for the video. I did some research and pygame actually had a built-in module where video could be presented within the same window. However, for whatever reason, the built-in module

(pygame.movie) had deprecated and is no longer supported. This meant that I had to find another way to add a video in. It was quite difficult to find a module that could present a video within a Pygame window, so I spent lots of time searching on websites like stack overflow and Pygame. I also wanted a video that could skip through if the user wanted to. I stumbled across the module OpenCV or commonly known as 'cv2'. This module enabled me to put a recorded video of my game into the Pygame window. Since the module is currently not recognised on the latest version of Pygame, my game is running on Pygame 3.10 purely for this reason.

This module also allowed me to have a type of user input so they can skip the video. In this case, to skip, you would press 's'. The file also includes a piece of code that will run if the video does not run for whatever reason. So, it will say "error, could not load the video" instead of the whole entire game crashing.

I was weary at first of using the module since the video does not run in a Pygame window. Because of this, I decided to run this window, close it, then open my game window. That way, it does not cause any trouble with my actual game.

Then I decided to place this code inside a file and attach it to my game file, so my game code is less confusing and easier to read.

MAKING BUTTONS

Before I decided how my video was going to work, I made a start on a button class since making a title screen is the next thing I needed. I knew making the video would use up most of my time and was already doing so at the time. If I could go back, I would have definitely started making the video last. I would have prioritised making the actual game. So, while looking for modules for my video, I also had a look on YouTube to make a button class. I came across 'Coding with Russ' which accurately had a button class that would work in my game.

Here is the video I used: [PyGame Beginner Tutorial in Python - Adding Buttons - YouTube](#)

File Edit Format Run Options Window Help

```

import pygame
pygame.init()

class Button():
    def __init__(self, x, y, image, scale):
        width = image.get_width()
        height = image.get_height()
        self.image = pygame.transform.scale(image, (int(width * scale), int(height * scale)))
        self.rect = self.image.get_rect()
        self.rect.topleft = (x, y)
        self.clicked = False

    def draw(self, surface):
        action = False

        pos = pygame.mouse.get_pos()

        if self.rect.collidepoint(pos):
            if pygame.mouse.get_pressed()[0] == 1 and self.clicked == False:
                self.clicked = True
                action = True

            if pygame.mouse.get_pressed()[0] == 0:
                self.clicked = False

        surface.blit(self.image, (self.rect.x, self.rect.y))

        return action

```

Originally, I put this class in my game file to check everything works. Once that was done, I was able to put this into a separate file and add it as a module and imported it into my game file. The video I used to make this class was really helpful since it also showed me how to accomplish all of this.

```
import pygame, Button, sys, Videomodule, random, time
```

→ imported
buttons
and
video
into game
file

Having a class for this was quite useful because I found myself constantly adding new buttons throughout the making of my game. So, to achieve this, all I had to do was:

1. Define the image I selected for the button.
2. Using the class, define it as an object and add in all of the attributes I wanted it to have.
3. And finally, call the variable into the screen of when I wanted the button to show.

And then from there, I added my 'if statements' as to what I want my program to do when the user clicks the button.

```
level = pygame.image.load("level_button.png").convert_alpha()
tutorial = pygame.image.load("tutorial_button.png").convert_alpha()
settings = pygame.image.load("settings_button.png").convert_alpha()
quit_game = pygame.image.load("quitgame_button.png").convert_alpha()

level_button = Button.Button(380, 140, level, 0.4)
tutorial_button = Button.Button(380, 280, tutorial, 0.4)
settings_button = Button.Button(380, 420, settings, 0.4)
quitgame_button = Button.Button(380, 560, quit_game, 0.4)

level1button = pygame.image.load("level1.png").convert_alpha()
level2button = pygame.image.load("level2.png").convert_alpha()
level3button = pygame.image.load("level3.png").convert_alpha()
level4button = pygame.image.load("level4.png").convert_alpha()
level5button = pygame.image.load("level5.png").convert_alpha()

level1 = Button.Button(116, 126, level1button, 0.3)
level2 = Button.Button(232, 242, level2button, 0.3)
level3 = Button.Button(348, 358, level3button, 0.3)
level4 = Button.Button(464, 474, level4button, 0.3)
level5 = Button.Button(580, 590, level5button, 0.3)
```

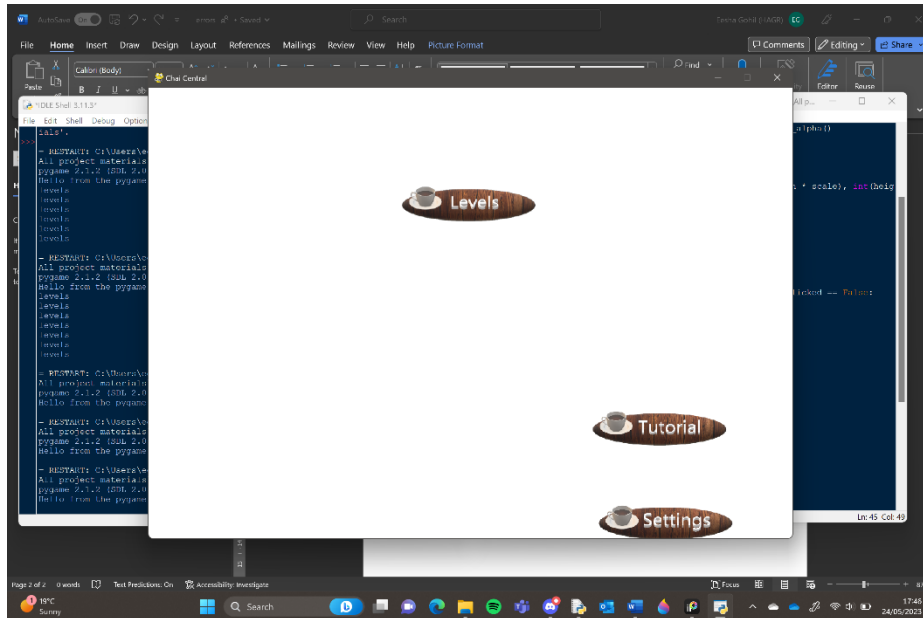
Here is where I defined my button variables. These are not all of them however this is just for the title screen and the levels screen. Here, the attribute of every button is (width, height, variable and size).

I made the button variable names "level1", "level2" etcetera so that anyone who views my code can easily understand what these variables are used for. Also, I have put comments into my code every few lines to indicate what some variables do. For example, the variables I have copied in here will all come under "button variables". I have put any images in as "image variables".

Candidate Name: Eesha Gohil

Candidate Number: 8065

I also realised that the placement of the buttons can be a little difficult to manage. Below is a screenshot of my buttons being set. I also had trouble putting the background on without it covering the buttons underneath. It did not take me long to realise that filling the screen first with a colour, then blitting the background would work fine. The buttons then had to be drawn on the back screen. In order to place the buttons in the place I wanted, I had to calculate the height and width of each button image against the height and width of my current screen. This was so I could ensure that each button image was in line with each other and at the centre of the screen.



To get mouse coordinates, I used this code inside the event loop, then I would print “x,y” out where I needed to. For example, like this: Finding placement of buttons on the main menu screen.

```
for event in pygame.event.get():  
    x, y = pygame.mouse.get_pos()
```

ALL BASIC VARIABLES 19/01/23

```

import pygame, Button, sys, Videomodule, random, time
pygame.init()

### screens, displays and images ###
screen = pygame.display.set_mode((1000,700))
screen2 = pygame.image.load("mainscreen.png").convert_alpha()
icon = pygame.image.load("icon.png")
pygame.display.set_icon(icon)
pygame.display.set_caption("Chai Central")
clock=pygame.time.Clock()
me = pygame.image.load("eeha.jpg").convert_alpha()
me1 = pygame.transform.scale(me, (244,242))
recipepage = pygame.image.load("recipes.png").convert_alpha()
recip = pygame.transform.scale(recipepage, (1000,700))
end_game = pygame.image.load("endgame.png").convert_alpha()
endgame = pygame.transform.scale(end_game, (1000,700))

### buttons ###
pointer = pygame.image.load("sugar.png").convert_alpha()
points = pygame.transform.scale(pointer, (412, 92))
speech = pygame.image.load("speechbubble.png").convert_alpha()
speechbubble = pygame.transform.scale(speech, (400, 187))
level = pygame.image.load("level_button.png").convert_alpha()
tutorial = pygame.image.load("tutorial_button.png").convert_alpha()
settings = pygame.image.load("settings_button.png").convert_alpha()
quit_game = pygame.image.load("quitgame_button.png").convert_alpha()

level_button = Button.Button(380, 140, level, 0.4)
tutorial_button = Button.Button(380, 280, tutorial, 0.4)
settings_button = Button.Button(380, 420, settings, 0.4)
quitgame_button = Button.Button(380, 560, quit_game, 0.4)

level1button = pygame.image.load("level1.png").convert_alpha()
level2button = pygame.image.load("level2.png").convert_alpha()
level3button = pygame.image.load("level3.png").convert_alpha()
level4button = pygame.image.load("level4.png").convert_alpha()
level5button = pygame.image.load("level5.png").convert_alpha()

level1 = Button.Button(116, 126, level1button, 0.3)
level2 = Button.Button(232, 242, level2button, 0.3)

```

Any of the lines in red is just to help me and anyone else viewing to understand where everything is. This technique also really helped me debug some of my errors. If I had an error with the buttons for example: Firstly, I knew it would not be the class since it is in a different file, and secondly, I can scroll up to all my button variables and find the error. I found it helpful to keep as many similar things in the same place as possible.

CONFIRM BUTTON 23/10/23.

Since I changed how the whole game would work throughout my timer working on this, I made a confirm button on screen quite late into the development. Below is a screenshot taken from one of the procedures used for the whole game which I explain into detail later. But essentially, the confirm button is how the user would check their answer over the randomly generated "recipe".

```

#checks user input over answer
if confirm_button.draw(screen):
    # tea2+i2 represents the user's answer
    if tea2+i2 == correction:
        num = num + 1
        count = count + 1
        t0 = t1
        t4 = t2
        print("correct")
        correction = ""
        tea1, tea2 = teastuff()
        i1, i2 = teastuff()
        peoples = random.choice(customerlist)
        customers = pygame.sprite.Group()
        customer = Customer(peoples, 445, 400)
        customers.add(customer)
        pygame.display.update()
    else:
        count = count + 1
        t0 = t1
        t4 = t2
        print("incorrect")
        correction = ""
        tea1, tea2 = teastuff()
        i1, i2 = teastuff()
        peoples = random.choice(customerlist)
        customers = pygame.sprite.Group()
        customer = Customer(peoples, 445, 400)
        customers.add(customer)
        pygame.display.update()

```

Adds a point 'sugar cube' for the user.

← adds the round

← resets the timer for the next round

← resets user answer for next round.

If answer is correct.

CLIENT OPINIONS

Successes:

- Buttons class is in a separate file hidden away from main code.
- Buttons included the same theme as the game (e.g. not rainbow, unfitting colours etc)
- Lots of buttons give the user a sense of choice.
- Button names are clear and understandable.

Failures:

- Some buttons take a while to register.

MAKING SCREENS

MAKING TITLE SCREEN 23/01/23

The title screen I made is quite simple. It is pretty much the same as to when I first added it into the game. But before I started, I had to make sure I knew where I wanted the destination of the program to be when the user clicks the button. Instead of figuring out the exact destination (example: the levels button should go to the levels screen) I added game states into my game.

My title screen also uses thinking logically to provide the user with different screens they can go on.

```
### main game loop ###
game_state = 1
run = True
while run:

    #menu screen
    if game_state==1:
        screen.fill((146, 144, 106))
        screen.blit(textsurface, (245,15))
        if level_button.draw(screen):
            game_state=2
        if tutorial_button.draw(screen):
            game_state=3
        if settings_button.draw(screen):
            game_state=4
        if quitgame_button.draw(screen):
            event.type==pygame.QUIT
            run = False
        pygame.display.update()

    #the main event loop
    for event in pygame.event.get():
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_ESCAPE:
                game_state=1
        if event.type==pygame.QUIT:
            run = False
    #makes the rect of input transparent on screen.
    i.set_alpha(128)
    i.fill((255,255,255))
    screen2.blit(i, (0,0))
    pygame.display.update()

pygame.display.update()
clock.tick(60)
pygame.quit()
```

clock is set to 60 so the frames can handle the timers I have behind the scenes.

Originally, this is all I had in my game loop. Before the while loop, I set game state to one so I could easily change the game state with numbers. So, during game state one, if the user clicks any of the buttons on the screen, it will take you to a different screen. For example, if the user clicks settings: the game state variable will change to two, which is the settings screen. Then here I have the event loop underneath the if statement for my menu screen. The event loop constantly checks for user input and if the user presses the red 'x' button on the window.

Test data for the title screen:

What am I assessing	Test type	predicted outcome	Actual Outcome
When mouse is outside of (x,y)= (1000,700)	erroneous	nothing	Nothing happens

When mouse is over a button	boundary	nothing	Nothing. However, using a touchpad on a laptop could accidentally trigger the "click"
When buttons are clicked	normal	Goes to different game state	Goes to a different game state.
When mouse.click on quit	normal	Game screen closes	Game screen closes
User input = "esc"(intended to allow the user to go back to menu screen)	normal	Does nothing	Does nothing. The "esc" button should do nothing at this point.

MAKING LEVELS SCREEN 26/01/23

After I made a title screen, I wanted each game state to be assigned to a different screen. So, the quit button was simple, I used `Pygame.quit()` like every other time I want my program to quit. Next thing I wanted to do was add a levels screen with all the various levels on it. I thought it would be easier for the user to understand where to go in order to play the game. I made the screen the exact same style as the title screen but instead it includes levels one through five. If I had more time and went on to develop this as a full game to put online, I would have made more levels and more code to increase the difficulty of the game.

```
#menu screen
if game_state==1:
    screen.fill((146, 144, 106))
    screen.blit(textsurface, (245,15))
    if level_button.draw(screen):
        game_state=2
    if tutorial_button.draw(screen):
        game_state=3
    if settings_button.draw(screen):
        game_state=4
    if quitgame_button.draw(screen):
        event.type==pygame.QUIT
        run = False
    pygame.display.update()

#levels screen
elif game_state==2:
    screen.fill((146, 144, 106))
    screen.blit(textsurface2, (375,15))
    if level1.draw(screen):
        game_state=5
    if level2.draw(screen):
        game_state=5
    if level3.draw(screen):
        game_state=5
    if level4.draw(screen):
        game_state=5
    if level5.draw(screen):
        game_state=5
    pygame.display.update()

#constant checks for user input
for event in pygame.event.get():
```

Here, I expanded from the making of my title screen and added the levels screen in the exact same way. At this point in time, I had made all the buttons lead to game state five.

This levels screen is my indication of thinking logically so this provides the user with a choice of what they want to do.

What am I testing	Test data type	Predicted outcome	Actual Outcome
User clicks a button	normal	Goes to game state five	Goes to game state five
User presses "esc"	normal	Goes back to main menu	Goes back to main menu

MAKING TUTORIAL LEVEL 06/12/23

Before making my actual game, I assumed making the tutorial would be easier however I was wrong. I decided to make my game first and go back to the tutorial level later. When I completed my main game, I was almost past the deadline. With more time, I could finish the tutorial level. Also with more research, I could have also figured out how to have a narrator going through the level at the top of the screen.

```

### tutorial sentences ###
sen1 = "Wagwan, welcome to Chai Central where you are your own barista!"
inp = "Click anywhere to continue once you've finished reading a sentence :)"
sen2 = "Don't worry! I'm here to teach u how to play!"
sen3 = "Ooo look our first customer!"
sen4 = "So, they've asked for a cup of tea.. in a code"
sen5 = "Your job is to decode it and click the right things ASAP!"
sen6 = "The code makes up letters that mean a certain object."
sen7 = "Head into settings to find out what each of them mean"
sen8 = "correct answers = 1 sugar cube. Incorrect answers = 0 sugar cubes"
sen9 = "Careful, It's not as easy as it sounds.."
sen10 = "Customers get angry if you fail or take too long, and certain tasks tak
sen11 = "Oh.. and codes may get longer, hehe..."

```

- aim to put these on screen and come one by one when user clicks the screen.

- was not able to 'un-draw' text on screen.

Here are the sentences that I tried to add into my final project that narrates how to play the game.

However, I did not have enough research in order to turn these strings into phrases on the screen. I also wanted it to be displayed on screen and controlled by user input. So, whenever the user clicks the screen, the text would change to the next step.

Here would be where I would ask the user to input their name and use it as a usability feature as I mentioned in the thinking ahead section.

MAKING THE SETTINGS SCREEN 07/12/23 1:27PM

Currently, there are no settings in my game. I had an idea where I would add things like volume controls, game controls and syncing progress to the cloud. However, my time management led me to never finishing this.

```

#settings screen
elif game_state==4:
    screen.fill((146, 144, 106))
    screen.blit(title, (310,15))
    pygame.display.update()

```

As shown in this screenshot, I only have the basic startup features for the screen which include the colour of the screen and the title to indicate to the user that this is the settings page. And updating the screen of course. I may add a picture of the game controls and blit it to the screen since with little time remaining, it is the least I can do.

CLIENT OPINIONS

Successes:

- Usability features are simplistic and easy to use.
- Buttons used have clear meanings of what you want.

Failures:

- Some screens do not work as intended. (As understood, may be still in the making)
- Usability may simply just be an image of what things do and not to user preference.

ATTEMPTING GAME STATE MANAGER 11/07/23

My original plan was to have lots of if statements in one game state. So, in game state five, if user clicked level 5: then make the game harder. Elif user clicked level 1: then do not add in the difficulty change code. However, I just did not think that would be beneficial and the structure of my code would become very messy. Later on, I changed all the game states to 5, 6, 7, 8 and 9. This decision also has its own errors that I had to deal with however the game still works. Before I even had a game running, I decided to attempt to make a class for my game states to improve the efficiency of my game and prevent it from being too slow. I used a video from “clear code” to try and manage my game states and perhaps even add a game state manager. Here’s the video I used: [Python / Pygame Tutorial: Creating multiple stages in a game - YouTube](#).

```
- RESTART: C:\Users\eesha\OneDrive - Harris Federation\year 12 computer science\
All project materials\recent_project.py
pygame 2.2.0 (SDL 2.0.22, Python 3.10.11)
Hello from the pygame community. https://www.pygame.org/contribute.html

- RESTART: C:\Users\eesha\OneDrive - Harris Federation\year 12 computer science\
All project materials\recent_project.py
pygame 2.2.0 (SDL 2.0.22, Python 3.10.11)
Hello from the pygame community. https://www.pygame.org/contribute.html

- RESTART: C:\Users\eesha\OneDrive - Harris Federation\year 12 computer science\
All project materials\recent_project.py
pygame 2.2.0 (SDL 2.0.22, Python 3.10.11)
Hello from the pygame community. https://www.pygame.org/contribute.html

- RESTART: C:\Users\eesha\OneDrive - Harris Federation\year 12 computer science\
All project materials\recent_project.py
pygame 2.2.0 (SDL 2.0.22, Python 3.10.11)
Hello from the pygame community. https://www.pygame.org/contribute.html

- RESTART: C:\Users\eesha\OneDrive - Harris Federation\year 12 computer science\
All project materials\recent_project.py
pygame 2.2.0 (SDL 2.0.22, Python 3.10.11)
Hello from the pygame community. https://www.pygame.org/contribute.html
Traceback (most recent call last):
  File "C:\Users\eesha\OneDrive - Harris Federation\year 12 computer science\All
project materials\recent_project.py", line 81, in <module>
    game_state.main_game()
AttributeError: 'int' object has no attribute 'main_game'

- RESTART: C:\Users\eesha\OneDrive - Harris Federation\year 12 computer science\
All project materials\recent_project.py
pygame 2.2.0 (SDL 2.0.22, Python 3.10.11)
Hello from the pygame community. https://www.pygame.org/contribute.html
Traceback (most recent call last):
  File "C:\Users\eesha\OneDrive - Harris Federation\year 12 computer science\All
project materials\recent_project.py", line 81, in <module>
    game_state.main_game()
AttributeError: 'int' object has no attribute 'main_game'
```

```
pygame.quit()
screen.fill((146, 144, 106))
screen.blit(textsurface, (245,15))
if level_button.draw(screen):
    self.state = "levels"
if tutorial_button.draw(screen):
    self.state = "main_game"
if settings_button.draw(screen):
    self.state = "settings"
if quitgame_button.draw(screen):
    event.type = pygame.QUIT
pygame.quit()

pygame.display.update()

def levels(self):
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
    screen.fill((146, 144, 106))
    screen.blit(textsurface2, (375,15))
    if level1.draw(screen):
        self.state = "main_game"
    if level2.draw(screen):
        self.state = "main_game"
    if level3.draw(screen):
        self.state = "main_game"
    if level4.draw(screen):
        self.state = "main_game"
    if level5.draw(screen):
        self.state = "main_game"
    pygame.display.update()

def settings(self):
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
    screen.fill((146, 144, 106))
    screen.blit(rtext, (447,34))
    screen.blit(menu, (63, 60))
    pygame.display.update()
    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_r:
            screen.fill((146,144,106))

def state_manager(self):
    if self.state == "start screen":
        self.start_screen()
    if self.state == "levels":
        self.levels()
    if self.state == "main_game":
        self.main_game()
    if self.state == "settings":
        self.settings()
    if self.state == "actual bulk":
        self.actual_bulk()
```

On the right-hand side here, I attempted to make a game state class. All the code in the screenshot currently is all the methods I tried to create for my object "game state". I could not get it to work and eventually ended up having many errors that I did not have time to research to fix. This error was not something small that I could go on and complete the rest of my game. As soon as the video would finish at the start of my game, the title screen would just be black and the shell would return an error, like the ones in the left-handed screenshot. Because of this error, I was persuaded to restart and continue with making my actual game before the deadline.

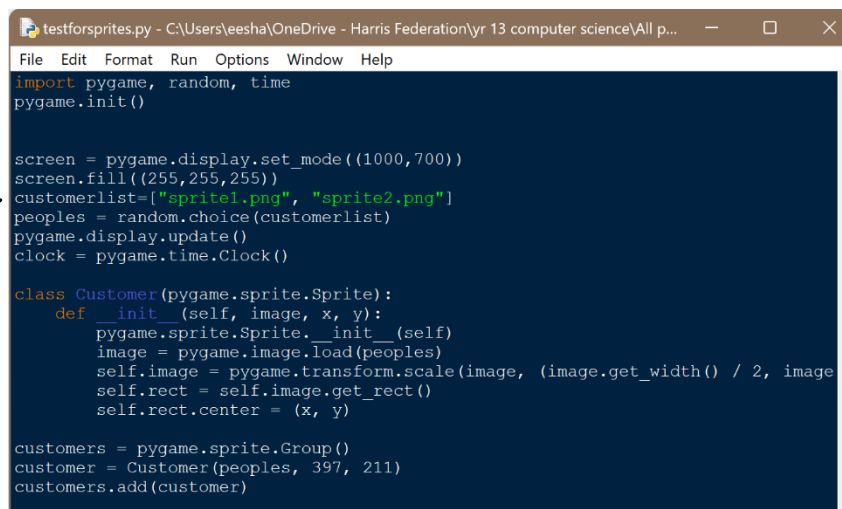
The main reason wanted to have a game state manager is because my method of using classes is too hard to maintain and can get really messy really quickly. I will also end up with too many if-elif clauses that affect the readability and performance of my code. All of which, I encountered inevitably.

CLASSES

CREATING A SPRITE CLASS 10/09/23

To make my sprite class, I used a separate file to make this just so it would be easier to find and fix errors.

list of images of
customers that is
randomly
chosen each
round



```
testforsprites.py - C:\Users\eesha\OneDrive - Harris Federation\yr 13 computer science\All p...
File Edit Format Run Options Window Help
import pygame, random, time
pygame.init()

screen = pygame.display.set_mode((1000,700))
screen.fill((255,255,255))
customerlist=["sprite1.png", "sprite2.png"]
peoples = random.choice(customerlist)
pygame.display.update()
clock = pygame.time.Clock()

class Customer(pygame.sprite.Sprite):
    def __init__(self, image, x, y):
        pygame.sprite.Sprite.__init__(self)
        image = pygame.image.load(peoples)
        self.image = pygame.transform.scale(image, (image.get_width() / 2, image
        self.rect = self.image.get_rect()
        self.rect.center = (x, y)

customers = pygame.sprite.Group()
customer = Customer(peoples, 397, 211)
customers.add(customer)
```

This screenshot is from the separate file 'test for sprites' where I actually assessed a lot of other classes. I watched another video to make an average sprite class using the Pygame sprite built in module. This class contains the image of the customer I created on paint 3d, and the height and width of each image. So far, I have only made two images of customers in the list with one more set of drawn customers on paint 3d in the making.

Going back to thinking abstractly, the customers do not have eyes for example or have the ability to speak. They are simply there to imitate the reality of a coffee shop. Adding the ability for them to talk or have facial expressions etc would be me coding unnecessary things because the sprites still function as intended. I did originally want eyebrow shape to change to help the user understand when a customer was getting impatient just like in reality. However, the timer seems to indicate this simply fine. Therefore, the usability for the game remains the same.

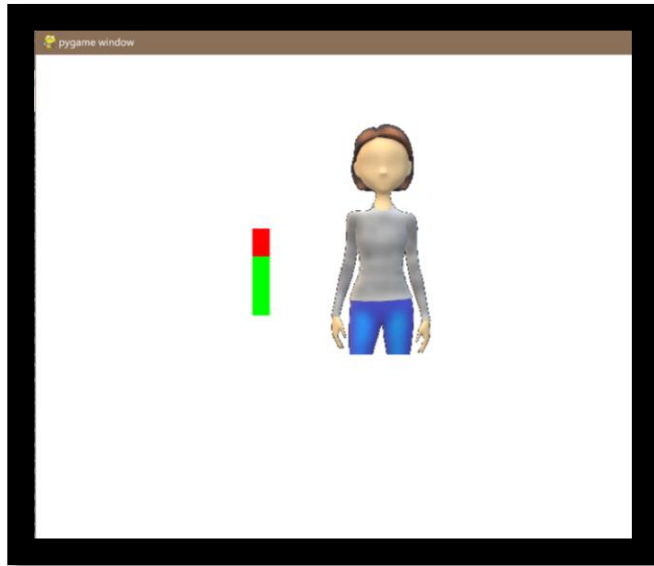
CREATING A TIMER CLASS 20/09/23

Using the same file for the customer class, I also have the timer class within it too.

```
class timer():
    def __init__(self, x, y, w, h, max_time):
        self.x = x
        self.y = y
        self.w = w
        self.h = h
        self.time = max_time
        self.max_time = max_time
    def draw(self, surface):
        ratio = self.time / self.max_time
        pygame.draw.rect(surface, "green", (self.x, self.y, self.w, self.h))
        pygame.draw.rect(surface, "red", (self.x, self.y, self.w, self.h * ratio))
```

timer = timer(250, 200, 20, 100, 5) ← calling the class with a timer of 5 seconds.

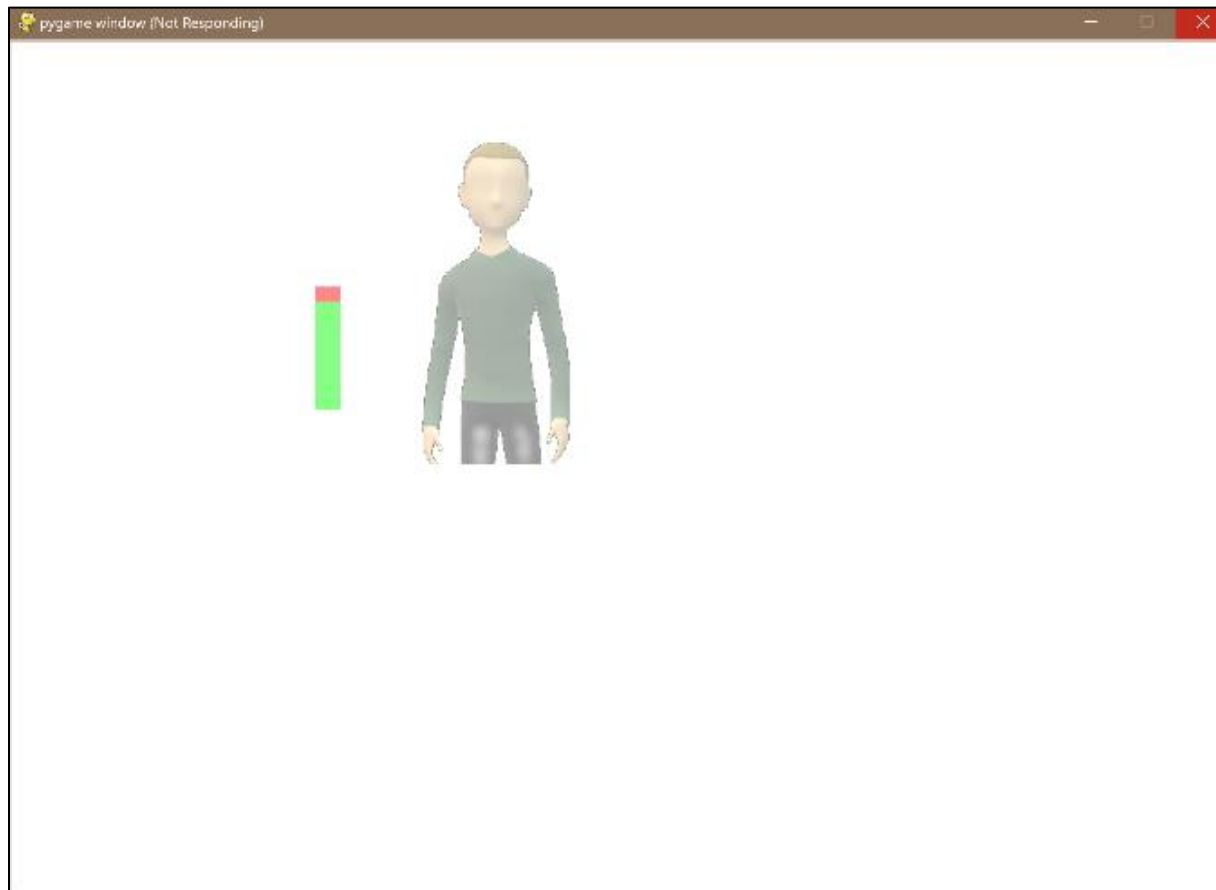
This class represents the timer displayed on the screen.



The screenshot is taken from the separate file, and this is also used from 'Coding With Russ'. However, since the video only uses this for a health bar, the green rectangle would only decrease in height when a sprite loses health. In this case, I want the green rectangle to decrease with time. So, I called the timer variable 'dt' as timer.time so when the time increases, the green rectangle decreases. As I will mention in the making a timer section, the timer I have implemented cannot be stopped once it starts so when the green rectangle finds the underlying cause of the red rectangle, the timer will reset, and a new round begins.

MAKING A TIMER 22/09/23

After I had made a screen for my main game, I soon realised that the game I decided to make revolves around a timer. It was difficult to put one in and I had lots of trouble implementing different methods of timers and none of them could do what I wanted it to do. The first timer I tried was using the built-in python timer module. Here is one of the main errors I encountered using `time.delay` and `time.sleep`.



This screenshot is from my test files. Here, I was evaluating how the anger bar would work next to a randomly generated sprite within my class. I used `timer.sleep` here as part of a built-in time module already inputted into python. The problem here is that `time.sleep` and `time.delay` etcetera will delay the entire program until the `sleep()`, or `delay()` has been fulfilled. Which is why here, the window resulted in an error since I tried to quit the program.

After this, I went onto the pygame website itself to research more about its time module. I found `Pygame.time.clock()` which sets a timer to zero after `Pygame.init` has been called then can be called again and taken away from the original time. I used this in my game for a brief period of time since it acted a lot like a stopwatch. However, the only issue I had with it was that it could not be stopped or restarted. In fact, every timer that I researched online could not be stopped meaning I constantly have something extra running other than the event loop behind the scenes. And this I assume can use up a lot of unnecessary memory and can make my game run slower.

The first method I explained made it to my final product because it meant I could start a timer and I could restart it whenever I wanted.

```
game_state = 1
t0 = time.time()
t4 = time.time()
run = True
while run:

    #creates a timer
    t1 = time.time()
    t2 = time.time()

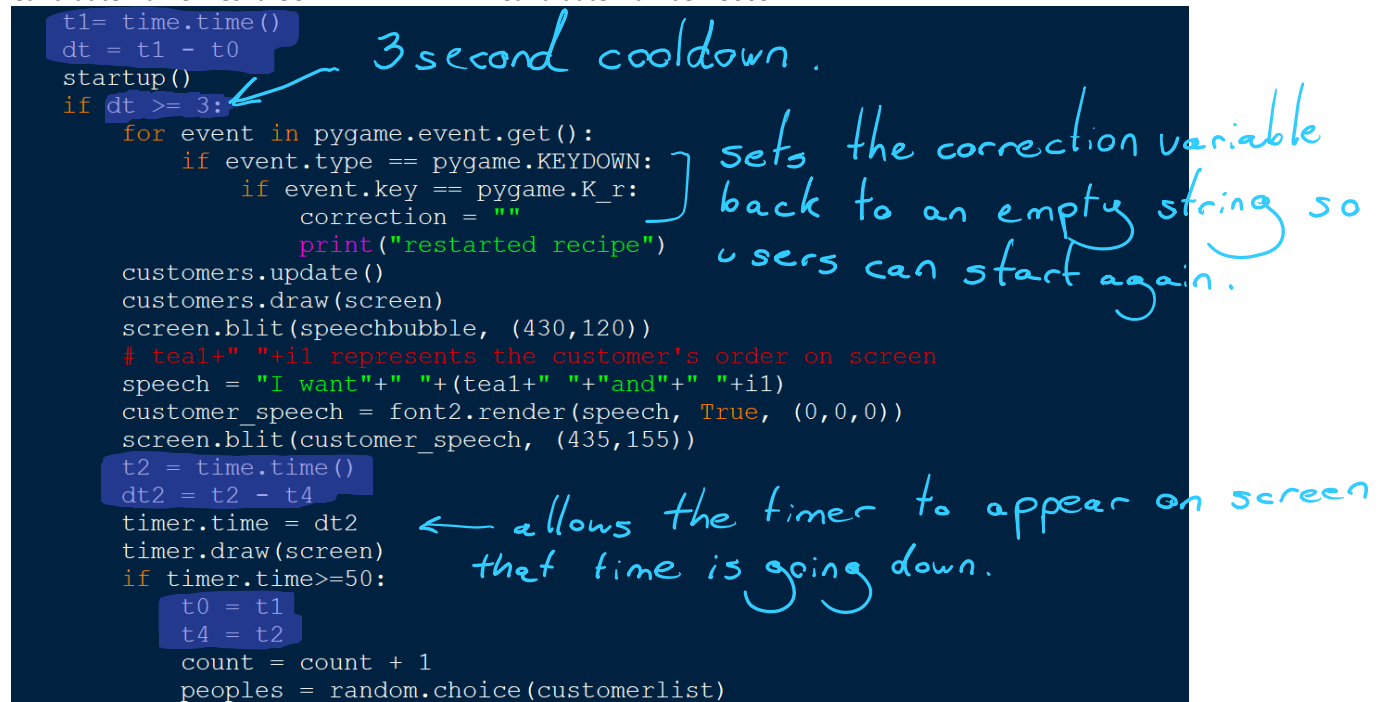
    #menu screen
    if game_state==1:
        screen.fill((146, 144, 106))
        screen.blit(textsurface, (245,15))
        if level_button.draw(screen):
            game_state=2
        if tutorial_button.draw(screen):
            game_state=3
        if settings_button.draw(screen):
            game_state=4
        if quitgame_button.draw(screen):
            event.type==pygame.QUIT
            run = False
        pygame.display.update()

    #levels screen
    elif game_state==2:
        screen.fill((146, 144, 106))
        screen.blit(textsurface2, (375,15))
        if level1.draw(screen):
            game_state=5
        if level2.draw(screen):
            game_state=6
        if level3.draw(screen):
            game_state=7
        if level4.draw(screen):
            game_state=8
        if level5.draw(screen):
            game_state=9
        pygame.display.update()
```

] defines the timers before the game loop

] officially starts both timers.

For my game, I needed two timers. One timer to display each customer after a fixed amount of time, and the other to display how much time the user had left to finish the round or level. So just above my game loop, I have defined two timers, and these are set to 0. Inside the game loop, I have defined two more timers which will go up in numbers as the loop iterates.



- The code written in this colour represents the code I used to create two timers. This shot is taken from a function I used to create my main game to then call again in the game loop. The first timer, represented by dt is used to create a constant time for when a customer first arrives on the screen. In my case, when you load up the game, this happens 3 seconds after: The user picks the level, the timer runs out, or the user presses confirm. These 3 seconds are vital since they act as a cooldown for the user to be prepared for the round coming next.
- The second timer represented by the variable dt2 is the timer displayed on screen. This tells the user how much time they have left before the round ends. The variable 'dt2' refers to the constant changing of the green rectangle on the screen compared to the red one.

CLIENT OPINIONS

Successes:

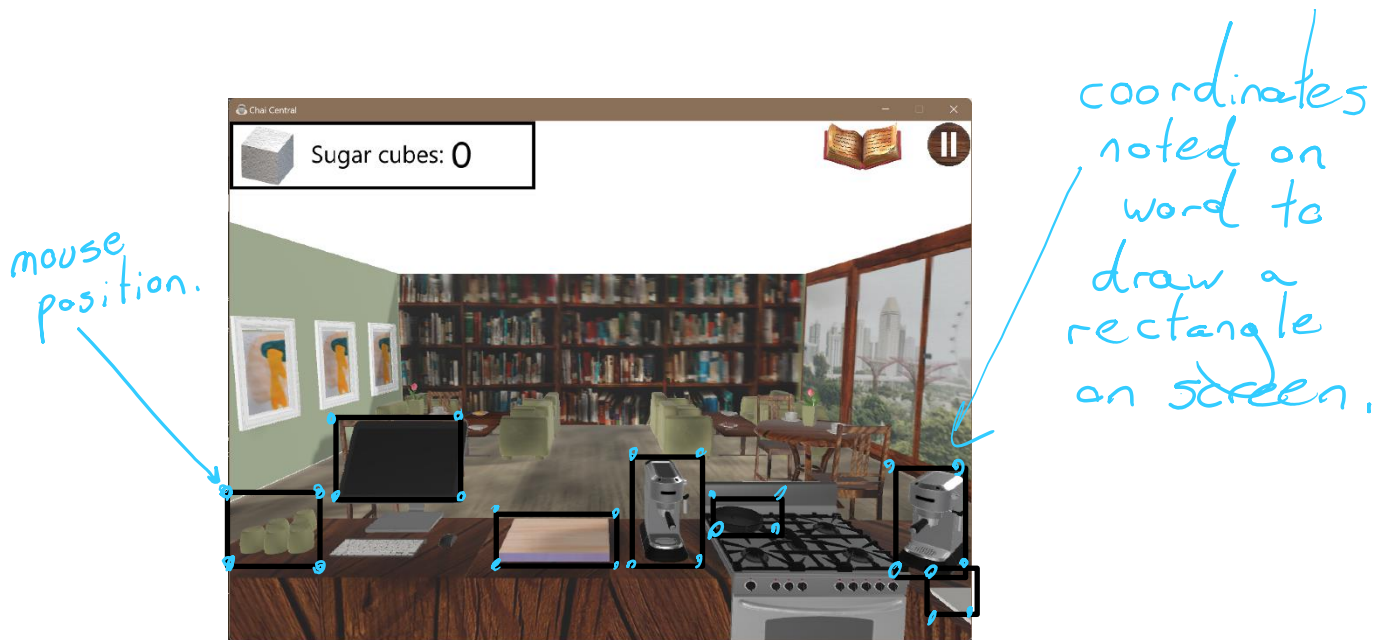
- Sprites on screen work as intended.
- Realism is portrayed by random customers arriving (alternating boy and girl)
- Timer works nicely giving the user a real game, imitating reality.
- Timer bar on screen works as intended and gives user an idea of time remaining.

Failures:

- Timers running in the background causes game to run slower.
- Sprites do not have much variety.

CREATING HIT BOXES 15/08/23

After creating a screen for my main game, I had to find a way for the user to click the objects on the screen. I had two options and that was: Either I import images over the screen to represent as objects or I put transparent boxes over the objects I have as the screen.



So, I chose to add transparent rectangle inputs on the screen instead since the screen would look more presentable and less messy. Here in the screenshot above, I have shown where I made my input boxes.

```
cm1 = pygame.Rect(551, 449, 76, 146)
cm2 = pygame.Rect(892, 459, 107, 155)
pan = pygame.Rect(643, 514, 83, 37)
cb = pygame.Rect(383, 534, 129, 59)
cup = pygame.Rect(29, 517, 83, 64)
cr = pygame.Rect(152, 400, 180, 112)
sink = pygame.Rect(944, 628, 55, 48)
```

Actual coordinates
to make each rectangle
around the objects.

```
i = pygame.Surface((146,76))
i = pygame.Surface((155,107))
i = pygame.Surface((37,83))
i = pygame.Surface((59,129))
i = pygame.Surface((64,83))
i = pygame.Surface((112,180))
i = pygame.Surface((48,55))
```

Position of all the input
boxes.

To make the boxes, I used 'mouse_pos' to constantly print out my mouse's coordinates on the screen so I could draw out the rectangles. Then I combined all the coordinates to make a rectangle around my objects. Then I used all the x and y coordinates of all the rectangles to place the transparency on the screen. I named all the variables 'i', so it was easy to implement transparency on all the rectangles simultaneously.

```
#the main event loop
```

```
for event in pygame.event.get():
```

```
    if event.type == pygame.KEYDOWN:
```

```
        if event.key == pygame.K_ESCAPE:
```

```
            game_state=1
```

```
    if event.type==pygame.QUIT:
```

```
        run = False
```

```
    #makes the rect of input transparent on screen.
```

```
    i.set_alpha(128)
```

```
    i.fill((255,255,255))
```

```
    screen2.blit(i, (0,0))
```

```
    pygame.display.update()
```

sets to full transparency →

brings user back to main screen when 'esc' key pressed.

i = all rectangle input box coordinates

Underneath the second comment is where the code is for the transparency of my rectangles as input boxes. It is all inside my event loop to ensure that the code is constantly in use every time the user clicks on a level (so when the main game function is called).

HIT BOXES: ONGOING ERROR

As I started to ask people to play my game, i noticed that when the user clicks on the objects, it does not seem to register well. You may have to click the object multiple times before it registers and activates the sound. I decided to change the mouse event just in case that was causing the error.

```
def main_game():
    global correction
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
        if event.type == pygame.MOUSEBUTTONDOWN:
            if cm1.collidepoint(event.pos):
                print("pouring coffee")
                coffee.play()
                correction = correction+"cm1"
                print(correction)
            if cm2.collidepoint(event.pos):
                print("pouring coffee")
                coffee.play()
                correction = correction+"cm2"
                print(correction)
```

in between these lines, I had:
'If event.button == 1'
so I removed this to allow all mouse input on the objects and not just left-click.

CREATING THE CONCATENATION OF USER INPUTS 23/10/23

Using the tuple that I had created; I was able to put them into the if statements for when the user clicks on an object. So, if user clicked sink, print "sink".

```
correction = ""
def main_game():
    global correction
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
        if event.type == pygame.MOUSEBUTTONDOWN:
            if event.button == 1:
                if cm1.collidepoint(event.pos):
                    print("pouring coffee")
                    coffee.play()
                    correction = correction+"cm1"
                    print(correction)
                if cm2.collidepoint(event.pos):
                    print("pouring coffee")
                    coffee.play()
                    correction = correction+"cm2"
                    print(correction)
                if pan.collidepoint(event.pos):
                    print("warming pan...")
                    sizzle.play()
                    correction = correction+"pan"
                    print(correction)
                if cb.collidepoint(event.pos):
                    print("chopping ingredients...")
                    chopping.play()
                    correction = correction+"cb"
                    print(correction)
                if cup.collidepoint(event.pos):
                    print("One cup taken")
                    cups.play()
                    correction = correction+"cup"
                    print(correction)
                if cr.collidepoint(event.pos):
                    print("Tea register")
                    keyboard.play()
                    correction = correction+"cr"
                    print(correction)
                if sink.collidepoint(event.pos):
                    print("washing...")
                    sinks.play()
                    correction = correction+"sink"
                    print(correction)
    pygame.display.update()
```

← concatenates to the user's answer. At the end, the answer should match the recipe required.

This procedure is used so I can print out what they click essentially and concatenate them together to form the recipe in which I explained earlier. This means when the user clicks on confirm, their concatenated answer of clicks can be compared to the one that was printed from the tea procedure. The tea recipe is compared to the variable 'correction' which has nothing in it currently at the top there.

To reset the correction variable, users can click the key 'r'. When the gamer gets the round correct after pressing confirm, the correction variable resets. If the gamer gets it wrong, the variable also resets. And lastly, if the timer runs out, the code classes it as the user failed the round and the variable resets yet again.

CLIENT OPINIONS

Successes:

- Sentences are clear to the user.
- Tea types are authentic to India making the game unique.

Failures:

- Some sentences do not fit in the speech bubble on screen.
- Pressing "r" may be unknown to the user.

SOUND EFFECTS 16/08/23

After creating my input boxes, the next thing I wanted to do was add sound effects, so it was evident to the user that clicking on the object worked.

```
### sounds ###  
  
ding = pygame.mixer.Sound("service-bell-ring-14610.mp3")  
ding.set_volume(0.4)  
coffee = pygame.mixer.Sound("coffee_V1.mp3")  
coffee.set_volume(0.4)  
ambience = pygame.mixer.Sound("ambience.mp3")  
ambience.set_volume(0.4)  
sinks = pygame.mixer.Sound("sinks_V11.mp3")  
sinks.set_volume(0.4)  
cups = pygame.mixer.Sound("cups_V1.mp3")  
cups.set_volume(0.4)  
sizzle = pygame.mixer.Sound("sizzle.mp3")  
sizzle.set_volume(0.4)  
keyboard = pygame.mixer.Sound("keyboards_V1.mp3")  
keyboard.set_volume(0.4)  
chopping = pygame.mixer.Sound("chops_V1.mp3")  
chopping.set_volume(0.4)  
elevator = pygame.mixer.Sound("jazz music.mp3")  
elevator.set_volume(0.4)
```

In order to do this, I decided to look for a simple YouTube video to help with sounds. I used copyright free sound effects to represent coffee sounds for example. So, I created this section at the top of my file for sounds, so they all have their place.

The code in yellow is something that I attempted adding to symbolise the ambience of the customers in the background. However, to do this, I had to put the sound in the game loop. When I did this, the sound sounded static in the background, and it would not be practical for the gamer. For this reason, I did not add it in to my final product. Even though it does not work, I still wanted to keep the ambience code in the file so I can do more research, so in the future I can add this in.

The code in pink is not actually a sound effect but instead like waiting music for when the user has paused the game. I thought this would be a funny little idea to add elevator music when the game is paused, just to add that bit of retro feeling into the game.

To add on, using thinking ahead from my previous section, I have included sound so the user knows when they click an item. Otherwise, you would be unable to tell. Here, the input would be from the user's mouse and the output would be the sound and the concatenation of the strings so the computer can keep track of the order the items have been clicked behind the scenes. I also had the idea of adding visuals so if the game is on mute, the game can still be played. However, I could not get round to that. I will explain more in the evaluation section.

CLIENT OPINIONS

Successes:

- Is a subtle way to increase realism of the game.
- Sounds work as intended and correspond to the correct items on screen.

Failures:

- Sound has a chance of bugging out.
- Sound effects can overlap if items are continuously clicked.

CREATING A TUPLE (AIM OF THE GAME)

ATTEMPT 1 23/10/23.

At this point I had started to make my actual game. The next thing I needed after sounds was a list. I knew that I needed at least one list for all the tea names, but I spent a while trying to understand how I could define a variable in a rectangle of input and a string at the same time. I tried defining a list as another list but due to the syntax of python, it is not possible.

```

File Edit Format Run Options Window Help
##phx = "phx"
##variable = [ysa, fzk, grv, dxo, rzm, iuy, phx]
##random.shuffle(variable)
##one = variable.pop()
##two = variable.pop()
##three = variable.pop()
##four = variable.pop()
##variable_name = one+two+three+four
##customer_speech = font1.render(variable_name, True, (0,0,0))

ysa = pygame.Rect(551, 449, 76, 146)
fzk = pygame.Rect(892, 459, 107, 155)
grv = pygame.Rect(643, 514, 83, 37)
dxo = pygame.Rect(383, 534, 129, 59)
rzm = pygame.Rect(29, 517, 83, 64)
iuy = pygame.Rect(152, 400, 180, 112)
phx = pygame.Rect(944, 628, 55, 48)

variable = [ysa, fzk, grv, dxo, rzm, iuy, phx]
number = random.randint(0,6)
rand = variable[number]
one = variable[number].pop()
two = variable[number].pop()
three = variable[number].pop()
four = variable[number].pop()
variable_name = str(one)+str(two)+str(three)+str(four)
customer_speech = font1.render(variable_name, True, (0,0,0))

variable = ["ysa", "fzk", "grv", "dxo", "rzm", "iuy", "phx"]

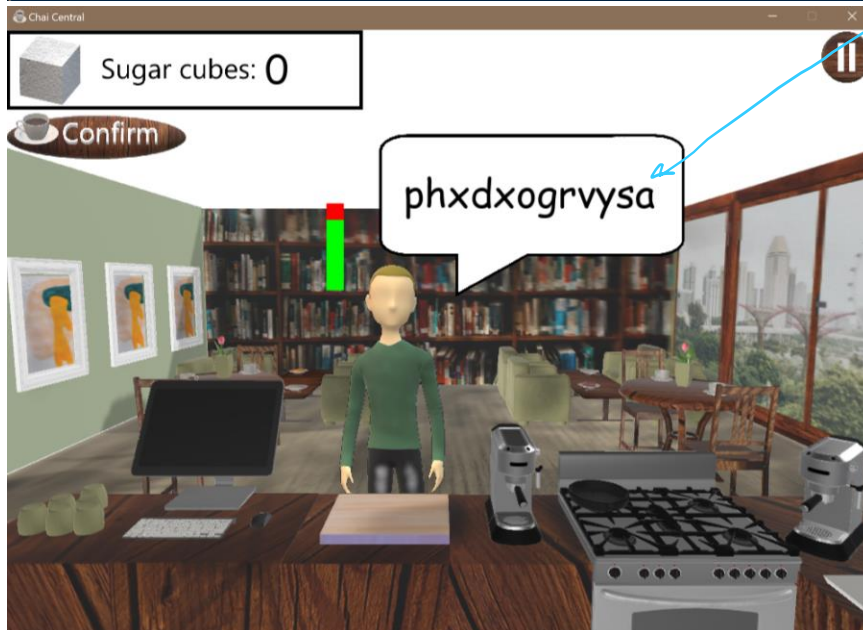
```

previous attempt that did not end up working at all.

Input

list of the object related to the prefixes.

list of prefixes of objects that can appear on screen



Output

This was my first attempt during the October half term holidays. I decided to make my game differently this time so a customer would arrive and ask for a bunch of 'gibberish'. The 'gibberish' was made up of three letter prefixes. I had randomly picked these using a random generator of letters with a word length of three. These little prefixes would represent one of the seven objects that users can interact with on screen.

To my surprise, the whole concept did work however using the help of beta testing from some friends from my year, this idea was not feasible and extremely hard to understand without an information page or tutorial.

The idea was that I define the object as a constant then I used that same constant as a string so then it could be displayed on screen as the sprite's speech. To put it simply: When the code is run, the prefixes are defined as string and put into another variable which is used for the sprite's speech. After this is done, the prefix variables are then changed to define the user input rectangle box instead of the strings.

I have screenshotted the code below so I can show that the way that I have sequenced it. Any other sequence would result in a logic error. This makes this idea not feasible since the code is volatile and can be easily jumbled if one line of code stops working.

```
ysa = "ysa"
fzk = "fzk"
grv = "grv"
dxo = "dxo"
rzm = "rzm"
iuy = "iuy"
phx = "phx"
variable = [ysa, fzk, grv, dxo, rzm, iuy, phx]
random.shuffle(variable)
one = variable.pop()
two = variable.pop()
three = variable.pop()
four = variable.pop()
variable_name = one+two+three+four
customer_speech = font1.render(variable_name, True, (0,0,0))
```

Randomly selects an element from the list and removes it completely until the next round in game.

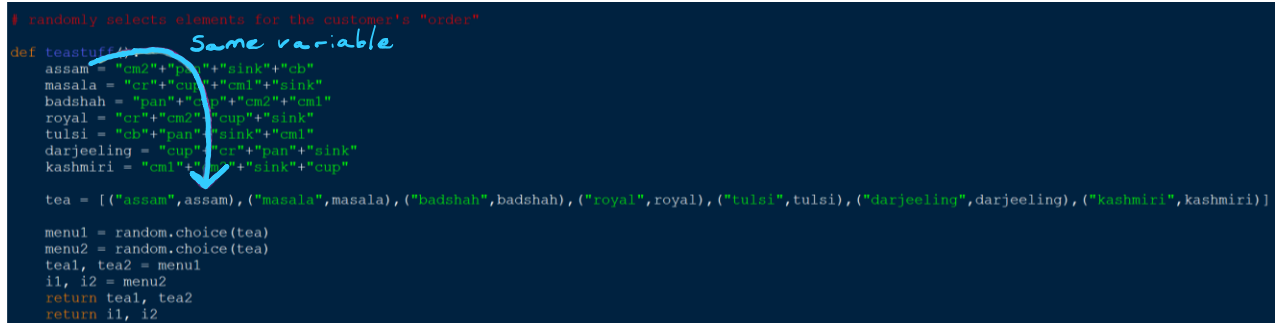
```
ysa = pygame.Rect(551, 449, 76, 146)
fzk = pygame.Rect(892, 459, 107, 155)
grv = pygame.Rect(643, 514, 83, 37)
dxo = pygame.Rect(383, 534, 129, 59)
rzm = pygame.Rect(29, 517, 83, 64)
iuy = pygame.Rect(152, 400, 180, 112)
phx = pygame.Rect(944, 628, 55, 48)
```

Prefixes being defined as input rectangle boxes after being defined as strings.

ATTEMPT 2 10/11/23

Using the feedback that I received from my peers, I decided to use the names of different teas from India into the game. Another peer of mine suggested I use a tuple which is exactly what I did. First, I created the different recipes of tea to use.

For example: To make assam tea, the user must click the second coffee machine, pan, sink, then the chopping board all in order under the time limit set. I did this by first defining all the tea recipes like that. Then I made the tuple itself which combined two things together as each element. This would include the name of that tea in the data type string and its corresponding recipe. And this I had already defined for all tea types earlier.



```
# randomly selects elements for the customer's "order"
def teastuff():
    assam = "cm2"+"p"+"s"+"sink"+"cb"
    masala = "cr"+"cup"+"cm1"+"sink"
    badshah = "pan"+"c"+"p"+"cm2"+"cm1"
    royal = "cr"+"cm2"+"cup"+"sink"
    tulsi = "cb"+"pan"+"sink"+"cm1"
    darjeeling = "cup"+"cr"+"pan"+"sink"
    kashmiri = "cm1"+"p"+"s"+"sink"+"cup"

    tea = [("assam", assam), ("masala", masala), ("badshah", badshah), ("royal", royal), ("tulsi", tulsi), ("darjeeling", darjeeling), ("kashmiri", kashmiri)]

    menu1 = random.choice(tea)
    menu2 = random.choice(tea)
    teal, tea2 = menu1
    il, i2 = menu2
    return teal, tea2
    return il, i2
```

I put all the variables and tuple related to this inside a procedure so it can be easy to call later in the while loop. It also saved me typing out the same code over and over again. After defining all of this, below that, I have some more variables. These variables control how they are printed.

Before putting this function in my game file, I used the test file to assess it first so I could see what was being printed first, and this method saved me lots of time opening my game, clicking the buttons etcetera to get to that point.

What the two variables return is this:

```

File Edit Format Run Options Window Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bi
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\eesha\OneDrive - Harris Federation\yr 13 computer science\A
l project materials\Project code copy.py
pygame 2.2.0 (SDL 2.0.22, Python 3.10.11)
Hello from the pygame community. https://www.pygame.org/contribute.html
Exit requested.
Traceback (most recent call last):
  File "C:\Users\eesha\OneDrive - Harris Federation\yr 13 computer science\A
l project materials\Project code copy.py", line 137, in <module>
    one = variable[number].pop()
AttributeError: 'pygame.Rect.Rect' object has no attribute 'pop'. Did you mean:
'top'?
>>>
= RESTART: C:\Users\eesha\OneDrive - Harris Federation\yr 13 computer science\A
l project materials\Project code copy.py
pygame 2.2.0 (SDL 2.0.22, Python 3.10.11)
Hello from the pygame community. https://www.pygame.org/contribute.html
Exit requested.
Traceback (most recent call last):
  File "C:\Users\eesha\OneDrive - Harris Federation\yr 13 computer science\A
l project materials\Project code copy.py", line 137, in <module>
    one = variable[number].pop()
AttributeError: 'pygame.Rect.Rect' object has no attribute 'pop'. Did you mean:
'top'?
>>>
= RESTART: C:\Users\eesha\OneDrive - Harris Federation\yr 13 computer science\A
l project materials\Project code copy.py
pygame 2.2.0 (SDL 2.0.22, Python 3.10.11)
Hello from the pygame community. https://www.pygame.org/contribute.html
Exit requested.
>>>
= RESTART: C:\Users\eesha\OneDrive - Harris Federation\yr 13 computer science\A
l project materials\Testing numbers.py
tulasi masala
chpansinkmcircrcpmsink
>>>

```

Represents what the customer will say on the screen.

This represents the combined recipe of both teas mentioned above this line.

After implementing this to my game, I just had to call it when I wanted throughout my while loop, and everything was in place.

SUB SCREENS

THE PAUSE SCREEN 09/09/23

The pause screen is the same as the recipe screen however I have keyboard controls on it instead of buttons. C to continue and Q to quit.

```
### all procedures, classes etc###

# pause screen
def pauses():
    elevator.play()
    paused = True
    while paused:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
                sys.exit()
            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_c:
                    paused = False
                    elevator.stop()
                elif event.key == pygame.K_q:
                    pygame.quit()
                    sys.exit()
        screen.fill((146, 144, 106))
        screen.blit(pausemenu, (245,15))
        screen.blit(pausemenu1, (190,200))
        #screen.blit(me1, (350,357))
        pygame.display.update()
        clock.tick(5)
```

The code is longer than the recipe screen because of the code for the keyboard controls and also filling the screen's colour and inputting text as opposed to an image of writing like I had done for the recipe book. For both the recipe screen and the pause screen, I have a clock tick of five to save the CPU memory from running a clock of sixty. The two procedures do not have anything going on apart from the screen so I would not want to spend all the CPU usage as it would ruin the game for users too.

The pause screen here relates back to my thinking abstractly section since I want users to be able to pause the game and continue when needed. Initially as explained also in that section, I wanted users to be able to see their progress, control volume and essentially help set the game up to their preference. If I had more time to do so, I would add those in and therefore have a functional settings screen as well.

CLIENT OPINIONS

Successes:

- Usability is provided for, can pause the game and timer stops.
- Successfully gives options to continue game or quit.

Failures:

- Could have more usability such as options to switch background music off or adding current progress to the screen.

THE RECIPE SCREEN 15/11/23

To help users understand how the game can be played, I have added a little recipe book icon at the top right of my screen. When clicked, it will pause the game and take you to another screen which shows all the different recipes of the different teas on there.



recipe book.
only available on screen from levels 1-3 to increase difficulty.

```
def recipebook():
    paused = True
    while paused:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
                sys.exit()
        screen.fill((0,0,0))
        screen.blit(recip, (0,0))
        if return_button.draw(screen):
            paused = False
        pygame.display.update()
        clock.tick(5)
```

Procedure for the recipe book screen. The timer stops when this screen is being displayed to make the game more playable. This section was easier to do since I already made a pause screen and applied the same bit of code for it. I left this part of my game as one of the last things I had to do due to the fact that I waited for feedback from my peers after they had played my game. It was only until after this addition that I found my game to be feasible.

CLIENT OPINIONS

Successes:

- Gives the user a “recipe book” of essentially how to play the game.
- Return to game button is provided at top right of screen.
- Timer stops while this screen is displayed.

Failures:

- May defeat the purpose of the game, as takes long to read.
- User has to keep returning to this screen to see the order.

CREATING PROCEDURES FOR THE MAIN GAME

The next step here for me was to combine all the trivial things I had done. In order to do this, I put everything into a procedure so it can easily be called again in the while loop. Here is the initial startup procedure for my main game procedure.

STARTUP PROCEDURE 30/10/23

```
# initial start up for the main game
num = 0
def startup():
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
    screen.fill((255, 255, 255))
    screen.blit(screen2, (0,0))
    screen.blit(points, (0,0))
    point = font1.render(str(num), True, (0,0,0))
    screen.blit(point, (299,12))
    if pause_button.draw(screen2):
        pauses()
```

When the user first clicks a level, I did not want absolutely everything to be drawn on the screen all at once. So, I made this little piece of code so it can be run without everything else. The main game is only called three seconds after because I want the customer to come in three seconds after the button is pressed. This enables the user to prepare for the start of the level. Here is the main game procedure.

GAME PROCEDURE 07/11/23

All this code represents a single game procedure. It works well however the long code is not feasible. I attempted many times to shorten down the code hence the startup procedure above this one. But I soon figured that the shortening of this code into many more procedures would be too difficult for other coders and myself to understand what is going on. So, I thought creating this was the best approach I had in mind.

Another issue that you can see below here is the vast number of global variables I used in this sub-routine. I did some research about the use of global variables on the internet and deduced that it is a bad practice due to it cluttering *'up the global namespace and are slower to look up than local variables.'* This was taken from Stackoverflow.

```
def game():
    global t0
    global t1
    global dt
    global customers
    global tea1
    global tea2
    global i1
    global i2
    global t4
    global t2
    global dt2
    global correction
    global num
    global count
    t1 = time.time()
    dt = t1 - t0
    startup()
    if dt >= 3:
        for event in pygame.event.get():
            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_r:
                    correction = ""
                    print("Restarted recipe")
                    customers.update()
                    customers.draw(screen)
                    screen.blit(speechbubble, (430, 120))
                    screen.blit(speechbubble, (430, 120))
                    # tea1, # i1 represent the customer's order on screen
                    speech = "I want " + t1 + " and " + t2 + " and " + i1
                    customer_speech = font2.render(speech, True, (0, 0, 0))
                    screen.blit(customer_speech, (435, 155))
                    t2 = time.time()
                    dt2 = t2 - t4
                    timer.time = dt2
                    timer.draw(screen)
                    if timer.time >= 50:
                        t0 = t1
                        t4 = t2
                        count = count + 1
                        peoples = random.choice(customerlist)
                        customers = pygame.sprite.Group()
                        customer = Customer(peoples, 445, 400)
                        customers.add(customer)
                        correction = ""
                        tea1, tea2 = teastuff()
                        i1, i2 = teastuff()
                    main_game()
                    # checks user input over answer
                    if confirm_button.draw(screen):
                        # tea1, # i1 represent the user's answer
                        if tea2 + i2 == correction:
                            num = num + 1
                            count = count + 1
                            t0 = t1
                            t4 = t2
                            print("Correct")
                            correction = ""
```

nc. global variables = bad practice.

after 3 seconds, load everything.

restarts user's answer

speech on screen.

starts a timer.

goes to next round.

calls customer class.

resets timer

```

customers.draw(screen)
screen.blit(speechbubble, (430,120))
# tea1 is *i1* represents the customer's order on screen
speech = "I want" + " " + (tea1 + "and" + " " + i1)
customer_speech = font2.render(speech, True, (0,0,0))
screen.blit(customer_speech, (435,155))
t2 = time.time()
dt2 = t2 - t4
timer.time = dt2
timer.draw(screen)
if timer.time >= 50:
    t0 = t1
    t4 = t2
    count = count + 1
    peoples = random.choice(customerlist)
    customers = pygame.sprite.Group()
    customer = Customer(peoples, 445, 400)
    customers.add(customer)
    correction = ""
    tea1, tea2 = teastuff()
    i1, i2 = teastuff()
main_game()
# tea2 is *i2* represents the user's answer
if confirm_button.draw(screen):
    # tea2+i2 == correction:
    if tea2+i2 == correction:
        num = num + 1
        count = count + 1
        t0 = t1
        t4 = t2
        print("correct")
        correction = ""
        tea1, tea2 = teastuff()
        i1, i2 = teastuff()
        peoples = random.choice(customerlist)
        customers = pygame.sprite.Group()
        customer = Customer(peoples, 445, 400)
        customers.add(customer)
        pygame.display.update()
    else:
        count = count + 1
        t0 = t1
        t4 = t2
        print("incorrect")
        correction = ""
        tea1, tea2 = teastuff()
        i1, i2 = teastuff()
        peoples = random.choice(customerlist)
        customers = pygame.sprite.Group()
        customer = Customer(peoples, 445, 400)
        customers.add(customer)
        pygame.display.update()

```

repetition of code represents a flaw in my code.

all the code below this symbolises the

The aim for this procedure is so I am able to put this entire piece of code into any of the if statements I put in which determines whether the user gets the point or not.

```

#level 1
elif game_state==5:
    t0 = t1
    t4 = t2
    count = 0
    while game_state == 5:
        game()
        if recipe_book.draw(screen):
            recipebook()
        if count == 3:
            game_state = 10
            break
        pygame.display.update()

#level 2
elif game_state == 6:
    t0 = t1
    t4 = t2
    count = 0
    while game_state == 6:
        game()
        if recipe_book.draw(screen):
            recipebook()
        if count == 3:

```

restarts two timers.

button code for the recipe book.

After game loops end, go to end screen.

In this screenshot, the code highlighted in yellow is the initial call up for the large procedure above this section. This meant I could repeat this as many times as I want outside the game loop. This meant that my overall game loop looked less cluttered.

So here, my game also includes thinking abstractly since the game is two dimensional. Originally, I wanted a three-dimensional look however it seemed to long and too extra to do, and there would be no benefit to the usability to my game.

Test Data:

What am I assessing	Test data type	Predicted outcome	Actual Outcome
Sprite does not spawn	invalid	error	Does not happen
User clicks game object	normal	Displays a certain sound for a certain number of seconds	Does exactly that
User clicks a non-game object	normal	nothing	Nothing
Game objects are performed simultaneously	erroneous	error	Can cause error in sound if use clicks objects without waiting for sound to end.
Sprite spawns while user is completing "sprite's order"	boundary	Rarely happens. true	Will not happen
User clicks too many game objects at once	boundary	error	Error occurs and game may lag for a while.
customer count=0 on a level.	normal	Game ends, finish screen pop up loads	Pop up loads and results are shown to the user with choices to go back or quit.
"esc" key to go back in screens	normal	Should go back to main menu when pressed at any time within the game.	"esc" key does not work at all. To quit the game, it must be paused before pressing q to quit.

CLIENT OPINIONS

Successes:

- Game works as intended
- Timers successfully remove and blit on the required images like sprites and speech bubbles.
- Round iteration and point system is accounted for and is counted.

Failures:

- Placement of hit boxes may be unknown to the user.
- No actual way to quit the game from here specifically since 'esc' button is unavailable. Would have to go to pause menu.

"ESC" KEY ERROR 01/12/23

This error I noticed a while ago but could not act on it since starting the main game was more important at the time. The aim of the "esc" key is to help the user return to the main menu, if for example, they changed their mind and they want to play a different level. The key works on every screen apart from the main game. I feel as if the issue is based on memory and my timers, not the actual code itself. Earlier into my game, before I added my timers, the escape key would work in this game state. Testing this after the development of the timers, it does not work.

Based on this, it could be the fact that the CPU is constantly performing and printing the seconds of the game. It became a large problem when I noticed that I could not stop the timers I had set once they started. This means that the CPU always has the extra task of counting seconds as soon as the game is run essentially. And this lowers the performance of everything else I need added to my game, in this case, the escape key.

ISSUE WITH USING UNNECESSARY GLOBAL VARIABLES 04/12/23.

Also, using Stackoverflow, they did include other method that I could use in order to approach this issue. They spoke about using functions and its return method. I did not want to overcomplicate my code and I was running out of time so this was the best I could do at the time. Stackoverflow mentioned many more issues with global variables:

- It can be easy to lose where you defined the variable and you might end up re-defining and overwriting this variable again, taking up unnecessary memory of the program.
- If the code is running of JavaScript, then this makes it harder to 'find' the variable you previously defined since JavaScript uses an interpreter to run.
- Can reduce the scalability and the testability of your code.
- Most of these issues are discussed further in this article which I found quite interesting and decided to read more about during the making of my program. This is the website I used. [Global Variables Are Bad \(c2.com\)](https://c2.com/articles/global-variables-are-bad)

THE END GAME (GAME STATE 10) 04/12/23

After I managed to create the main game for this project, I realised that there needed to be a point where the game ends after the certain number of rounds in that level. So, I quickly put together the end screen last minute. I decided to add this screen as another game state so I could easily keep track of all the states I have.

```

#endgame screen
elif game_state == 10:
    screen.fill((0,0,0))
    screen.blit(endgame, (0,0))
    cubes = font2.render(str(num)+"-"+str(count)+" sugar cubes!", True, (0,0,0))
    screen.blit(cubes, (584, 138))
    pygame.display.update()
    for event in pygame.event.get():
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_e:
                game_state = 2
            if event.type==pygame.QUIT:
                run = False

#the main event loop
for event in pygame.event.get():
    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_SPACE:

```

prints the
score you
got

The code for this screen comes just under the event loop since I attempted to code all the states in order of number. I aimed to make my code more understandable, so I made sure I laid out my code this way. As mentioned before, at the start of the file, I have all my defined variables like fonts, sounds etcetera. But here, I put one of the text variables here just to stop creating confusion about the variables. This code is quite far down compared to all my defined variables.

The end game screen will display how many points was scored as mentioned in the thinking procedurally section. Thinking concurrently also allows my game to keep running while the scoreboard can keep track of the user's progress.

CLIENT OPINIONS

Successes:

- Successfully provides the end to a level.
- Provides the final score
- Gives the option to quit or change level.

Failure:

- Would want to see more statistics on screen based on the final end of your performance in the rounds.
- More buttons or options can be used to go back to main screen for example.

ISSUE WITH THE SCOREBOARD 06/12/23

After the testing of the points (sugar cubes), I noticed an error with it. If you were to complete a level 3/3 correct and go to a different level, it would add up the points from the last level. So, it would say 5/3 sugar cubes won. During writing this, I thought I was unable to fix this issue. I tested out putting "num = 0" before all the levels start and that seems to fix the issue.

```
#level 5
```

```
elif game_state == 9:
```

```
    num = 0
```

```
    t0 = t1
```

```
    t4 = t2
```

```
    count = 0
```

```
    while game_state == 9:
```

```
        game()
```

```
        if count == 5:
```

```
            game_state = 10
```

```
            break
```

```
    pygame.display.update()
```

← placed this so points restart after levels are clicked.
E.g: this is level 5. I have done this for all my levels.

Even though my strategy fixed the issue, it means I cannot go ahead with the saving data idea because there is now nothing to save. There may have been better ways to prevent this logic error however with such little time remaining, I decided that this would be the only reasonable way. More research would have to be done for this.

The sugar cubes are used as a point system at the top left of the screen for usability which is mentioned in thinking procedurally. In some parts of this document, you may see coins or something different. This is because I had last minute changed my idea of currency since I wanted to make it different to the games I had researched.

D. EVALUATION

FINAL THOUGHTS – WITH CLIENT

As a result of my code, I now have a list of what needs to be changed and what needs more research in order to get to my endpoint. With a conversation with the client, these are the criteria that have been met and what needs to be met.

CRITERIA THAT HAS BEEN SUCCESSFUL

- My game states and how the keys work.
- My main game is somewhat functional and ends when listed.
- I have a short intro video of the game's setting which is the little coffee shop I made on 3d paint.
- The user is able to pause and quit the game during gameplay.
- The layout of my game and designs are all linked to the theme of it all.

- I was able to add sound effects and music into my game.
- The recipes of different teas all add up to what the customer asks for.
- Sprites arrive and leave when I want them to. Sprites are also randomly generated on screen for the two customisations that I have. I have more customisations in the making currently.
- My timers work and start when I want them to. Even though timers cannot be stopped or removed, I am able to restart them back to the time zero.
- The point system seems to work and goes up by one whenever the user gets the order correct.
- When the user gets the order incorrect, timers restart and goes onto the next round.

MAINTENANCE ISSUES/ LIMITATIONS

These are issues I consider points that **affect gameplay and will need development in the future**. These points are also the ones that prevent my game from looking its best and **reduces professionalism**.

- When an object is pressed, the user should also be able to see some sort of visual effect alongside my already inputted object sounds. My client mentioned that if sound is off, it is impossible to tell whether objects have been pressed or not. This is a valid issue in which can be solved in the way of visual effects. This makes usability poor for people who are deaf for example.
- I used an unsuitable number of global variables which mentioned before is bad practice.
- Background music I tried adding into my game sounded robotic.
- Speech in the speech bubble sometimes is too long and will not fit in.
- Lots of code has been repeated.
- While the game is loading, screen goes black for a couple seconds.

IMPROVEMENTS ON LIMITATIONS

- The items could be outlined with a sharp contrast colour in the future so users know what can and cannot be pressed.
- I would like to understand classes a lot better so I can implement them into my game and therefore remove the number of global variables I currently have. Classes would also remove the unnecessary lines of code I keep repeating throughout the program too.
- Background music would make the game realistic even though it is not needed. However, it is part of the criteria that my client wanted hence I would need to find another way to add music for this reason. Research would be key for this.
- For speech to fit in the bubble, I could resize it on the screen so more sentences can fit. Alternatively, more code could be added to make it so any sentences larger than 3 words for example, will split to start filling the next line under it.

- The screen may go black for a couple seconds at startup due to the large amount of background things going on. For example, I constantly have a timer running as soon as the program runs so I can keep track of the clock when the main game starts. More research will have to be done with timers so I can prevent lots of background tasks from happening. Perhaps researching a new module for it.

UNMET OR PARTIALLY MET CRITERIA

All these points are ones that I discussed at the very start with my client to be completed and added to the game. They unfortunately were unable to make it to the final product due to **lack of research and time**. Either this or the point did make it to the finals however **does not fully work as intended**.

- The tutorial level was unsuccessful.
- My settings screen is underdeveloped.
- My attempt on the game state manager was unsuccessful.
- Hit boxes work but sometimes do not register.
- When the sprite displays its order, for example: badshah and assam. The user will have to complete the order as it has been said. So, the program classes it as wrong if the user clicks for ingredients for assam and then badshah.
- Scoreboard works however code has been repeated.

SOLVING UNMET / PARTIALLY MET CRITERIA

Some of these criteria did not make it to the final product and with some **more research**, these can be completed. For example:

- Tutorial level can be completed.
- Settings screen can be completed.
- Game-state manager can be implemented because I currently have too many game states.
- Hit boxes can be re-made so error occurrences are rare.
- Order in which the teas are “brewed” is not accounted for and can still be marked as correct by the program.
- Scoreboard can be re-made so it does not use up many repeated lines of code in various areas of the program.

MAIN SUCCESS CRITERIA

Criteria	Justification	Did this idea make it to the final product?
It should display a menu screen	The three different buttons I wanted, play, settings, and the tutorial. This prevents the game from being run immediately and the player can choose what they want to do first.	Yes, however the tutorial has not yet been completed so more research would have to be done to create a level for that.
Customer's patience	Gives you a time for the order to be completed. Without this, the game is unachievable and non-realistic, there would be no point in the game overall.	Yes, works exactly the way I wanted it to. Is displayed on screen as a rectangle box and the rectangle goes down in size as the timer starts to run out.
Score (how much money you have as a whole)	Every order you complete, you will get ten coins. Every order you fail, you will lose ten coins.	I went for sugar cubes as the type of pointing system to make it more related to a chai making game. The user gains one instead of ten.
Item interaction on screen	Different items interact in diverse ways. So, I want the user to be able to do this. For example, mixing requires circular motions on the screen or dicing ingredients require a tapping motion on the screen.	Yes, this is in place. I have noticed that the user click does not register sometimes when clicking in the rectangle of input on the screen. I will have to do more research for this.
Settings	So, the user can edit and modify the game to their liking, such as volume and sound effects.	Was not able to achieve this in the time given so I will conduct more research for this.
Array of dishes	So, the user has to make and remember how to create a variety of dishes, increasing the difficulty of the game.	I changed the game to several types of tea instead of dishes. Instead of an array, I have a tuple for them. I also decided for this to not be the main idea to increase difficulty as I had other ideas.
Stopping the iteration of the game	The player gets three chances to continue. If you fail three orders within the level, the level automatically stops and displays the game over screen.	I did not add this idea since I thought it would be much simpler for the game to just iterate through with no points

		added to the score if user input is incorrect or wrong.
Random library	So, the dishes can be randomised so every level is not the same. Or even if you retry the level, the dishes to make is not the same.	I made the different teas into a tuple where the random library will randomly pick two of them from it. It is working and in place.
Customers on screen	Should arrive one at a time and only leave the screen when their order has been completed.	This is in place.
Sound effects	Such as customer noises, people in the background like reality, like a real restaurant to enhance the game's features.	Sound effects were only used for the objects blitted onto the screen. For example, the chopping board or the coffee machine.
Background music	As suggested by my stakeholder and also to enhance the features of the game.	I attempted this however the sound was very robotic. Will definitely do more research about that.

SUCCESS CRITERIA DETAILED

Menu screen

What am I assessing	Assess type	predicted outcome	Actual outcome
When mouse is outside of (x,y)= (1000,700)	erroneous	nothing	nothing
When mouse is over a button	boundary	nothing	nothing
When buttons are clicked	normal	Goes to different game state	Goes to different game state
When mouse.click on quit	normal	Game screen closes	Game screen closes
User input = "esc"(intended to allow the user to go back to menu screen)	normal	Does nothing	Does nothing

Video at the start of the screen

What am I assessing	Assess data type	Predicted outcome	Actual outcome
When game runs, video runs	normal	Video runs	Video runs
User input = "s"	normal	Skips video to main menu	Skips video to main menu

Levels screen

What am I assessing	Assess data type	Predicted outcome	Actual outcome
User clicks a button	normal	Goes to game state five	Goes to game state five
User presses "esc"	normal	Goes back to main menu	Goes back to main menu

Main game

What am I assessing	Assess data type	Predicted outcome	Actual outcome
Sprite does not spawn	invalid	error	error
User clicks game object	normal	Displays a certain sound for a certain number of seconds	Sometimes works but sometimes user will have to click more than once.
User clicks a non-game object	normal	nothing	nothing
Game objects are performed simultaneously	boundary	Should still register through	error
Sprite spawns while user is completing "sprite's order"	erroneous	error	Does not happen
User clicks too many game objects at once	boundary	error	Causes error if program feels as if user is spamming mouse clicks.
customer count=5/7 on a level.	normal	Game ends, finish screen pop up loads	Does as it should
User presses confirm button (determines point increase)	Boolean	Point system goes up by one or points stays the same	Does as it should

Tracking user input (when clicking an object (part of the tea recipe))	string	Converts user input into a string that concatenates to an empty one.	Does as it should
Pressing recipe button	Boolean	Should change screen to recipe screen and pause the time.	Does as it should
Pressing pause button	Boolean	Changes to pause screen and should pause the game.	Does as it should
Checking keyboard presses in pause screen: "c" or "q"	Boolean	C to continue game or q to quit the whole game.	Does as it should
"esc" key to go back in screens	normal	Should go back to main menu when pressed at any time within the game.	Error. Does not work at this stage.
End game to press "e" to go back to main menu	normal	Goes to main menu	Does as it should

CRITERIA OF USABILITY

THE MENU SCREEN SUCCESS CRITERIA

The first thing I assessed was the mouse on the screen. This was all normal and working since I first checked using the `mouse.pos()` function. It was especially important that I had the mouse being able to go across the screen without it pressing any buttons. Just to test the main functions of the game. During the creation of my input rectangle boxes, there was an issue where the input rectangle could be clicked on the main screen, so the boxes overlapped on the screen. Before I started making the actual game, I thought it should be important to make sure the overall usability features were all fully working. I also wanted to make sure that clicking the quit button actually worked. The last thing I checked was that the escape key worked so the user can go between different game states and always return to the main menu. And this works and made it to the final product.

THE VIDEO AT SUCCESS CRITERIA

The video actually came out surprisingly well. Although it was not what I wanted initially, it was the closest I could get to without spending lots of my time on it. I tried many other modules to make the video however none of them could skip the video using user input. The `cv2` module I used, allowed me to do that. A part of the video code actually codes an output for if the video does not run unexpectedly. Overall, even though I could not achieve the video loop under the button surfaces, I think the outcome I have now works just as well and demonstrates the graphics I used as a 360 perspective.

LEVEL SCREEN AT SUCCESS CRITERIA

The level screen is not much different to the menu screen. It includes the testing of the escape key if the user wants to go back to the main menu, and the checking of the user clicking each button and making sure they go to their different game states.

MAIN GAME AT SUCCESS CRITERIA

During the testing of the game, my sprites have never not spawned so if such was to happen, it would be an error. I tested the game objects during the first time I added it into the game, and everything seemed to work fine. However, as I have started to add more game states and memory into my game, the game object sometimes needs to be pressed more than once to actually be registered by the computer.

And this is a slight error that I do not know what the cause is. If more time were available to me, I could do some more research about why this is the case.

Clicking the game objects multiple times could eventually cause errors with the sound, however apart from that, the game objects seem to work as normal, even if the same item is clicked rapidly.

Before I properly decided to make my game and all of its attributes, I wanted more than one sprite to spawn on screen to indicate that another customer is in the queue of waiting for their order. Now that I have completed the game, if this happens, it is now seen as an error. I have purposefully made it so the sprites are only able to spawn 3 seconds after the timer restarts. So, this is when the user presses confirm on their order, or the timer runs out.

RECIPE / PAUSE SCREEN

My add-ons are the extra subroutines I added for usability specifically for my game. This is the pause screen and the recipe screen. These of which can be accessed by clicking the buttons at the top right of the main game screen. Here are my thoughts:

Recipe screen Failures	Recipe screen Successes	Pause screen Failures	Pause screen Successes
Could be seen as quite stressful with the word count on the page.	Button can easily be clicked during gameplay.	Background music may be distracting to some users.	Button can easily be located and pressed during gameplay.
Abbreviations corresponding to	The timer is paused while on this page.	Does not show any usability such as	Has the option for the user: game can be continued or quit.

item on main screen may be unknown.		current progress or control on settings.	
-------------------------------------	--	--	--

ROBUSTNESS

In this section of the evaluation, I will talk about how my game may respond to vigorous gameplay.

What could result in an issue	What happens as a result of it
Hit boxes have a chance of not registering	The hit box may need to be pressed again and may ruin the aim of the game.
An unhealthy number of clicks on the screen from the user.	Even though this sounds like an issue, it will not affect the timer. However, I noticed that it could result in the “confirm” button and the sprite flashing on the screen for a short amount of time. Should not affect gameplay though.
An unhealthy number of clicks to the hit boxes	This may affect the sounds of the game and can result in buggy sounds.
Continuously pausing the game and continuing the game.	This has the chance to result in the background music for the pause screen to sound static.

LET’S PLAY (VIDEO) W/ TIMESTAMPS

Here is my let’s play video link

[Computer Science coursework \(Chai Central\) \(youtube.com\)](#)

Here is the overview of the video and the timestamps of what happens in each section:

At the very start of the video, the video plays. For usability, at the bottom left of the screen, you can see a “press ‘s’ to skip...”. The user has the choice to skip the video or watch the rest of the video. In the video, the whole video is played to show my entire game.

At timestamps 0:10, the starting screen shows which provides the user with some choice of where they want to go next.

At 0:14, the levels are presented to the user. Levels 1, 2 and 3 are the easier levels and levels 4 and 5 are slightly harder.

At 0:19, I have clicked on level 1 and the main screen runs after a set time of 3 seconds after the level has been chosen. The customer also appears with their order and a timer starts to run which is the green rectangle next to the sprite. The green rectangle represents a timer of 70 seconds. The green rectangle decreases in height to represent time remaining each second going by.

The screen at 0:23 is the recipe screen. As shown, you have to click on the book for this state to appear. Bear in mind that this option or button is not available in levels 4 and 5 to increase difficulty of the game. Also to increase difficulty on levels 4 and 5, they have five rounds instead of three.

From the time 0:26 to 1:17, is where I demonstrate how to correctly complete a round. The recipe shows you the order of the items that should be pressed. Each item pressed has a different sound effect as presented in the video. And then you repeat for the next type of tea. When you have finished the order, you click the confirm button as shown. This lets the program know to check your progress and indicate your results. The result to you is shown at the top left of the screen in "sugar cubes". Every time you get an order correct; you gain a sugar cube.

At 1:18 shows the break of 3 seconds you get after the completion of a round.

At 1:20, I show that clicking random items and pressing confirm still completes the round but unsuccessfully. This is shown by the sugar cubes not going up by one in the top left corner.

At 1:43 as shown in the video, I show what happens when the timer runs out as part of the last round in the level before the game ends.

1:54 shows the end game screen displaying how many points you received and the option to quit the game or go back to the main menu.

At 2:01 I show my unfinished settings screen which will be developed in the future.

At 2:04 I present my tutorial game state which also needs more developing.

At 2:08 is my pause menu with my silly face on it from when I was younger and some background elevator music.

And finally, at 2:20, I quit the game.

PROJECT APPENDIXES**CODE LISTINGS**

```
import pygame, Button, sys, Videomodule, random, time
pygame.init()

### screens, displays and images ###
screen = pygame.display.set_mode((1000,700))
screen2 = pygame.image.load("mainscreen.png").convert_alpha()
icon = pygame.image.load("icon.png")
pygame.display.set_icon(icon)
pygame.display.set_caption("Chai Central")
clock=pygame.time.Clock()
me = pygame.image.load("eeha.jpg").convert_alpha()
me1 = pygame.transform.scale(me, (244,242))
recipepage = pygame.image.load("recipes.png").convert_alpha()
recip = pygame.transform.scale(recipepage, (1000,700))
end_game = pygame.image.load("endgame.png").convert_alpha()
endgame = pygame.transform.scale(end_game, (1000,700))
setting_pic = pygame.image.load("settings screen.png").convert_alpha()
settingpic = pygame.transform.scale(setting_pic, (950, 650))

### buttons ###
pointer = pygame.image.load("sugar.png").convert_alpha()
points = pygame.transform.scale(pointer, (412, 92))
speech = pygame.image.load("speechbubble.png").convert_alpha()
speechbubble = pygame.transform.scale(speech, (400, 187))
level = pygame.image.load("level_button.png").convert_alpha()
tutorial = pygame.image.load("tutorial_button.png").convert_alpha()
settings = pygame.image.load("settings_button.png").convert_alpha()
quit_game = pygame.image.load("quitgame_button.png").convert_alpha()

level_button = Button.Button(380, 140, level, 0.4)
tutorial_button = Button.Button(380, 280, tutorial, 0.4)
settings_button = Button.Button(380, 420, settings, 0.4)
quitgame_button = Button.Button(380, 560, quit_game, 0.4)

level1button = pygame.image.load("level1.png").convert_alpha()
level2button = pygame.image.load("level2.png").convert_alpha()
level3button = pygame.image.load("level3.png").convert_alpha()
level4button = pygame.image.load("level4.png").convert_alpha()
level5button = pygame.image.load("level5.png").convert_alpha()
```

```
level_button = Button.Button(380, 140, level, 0.4)
tutorial_button = Button.Button(380, 280, tutorial, 0.4)
settings_button = Button.Button(380, 420, settings, 0.4)
quitgame_button = Button.Button(380, 560, quit_game, 0.4)

level1button = pygame.image.load("level1.png").convert_alpha()
level2button = pygame.image.load("level2.png").convert_alpha()
level3button = pygame.image.load("level3.png").convert_alpha()
level4button = pygame.image.load("level4.png").convert_alpha()
level5button = pygame.image.load("level5.png").convert_alpha()

level1 = Button.Button(116, 126, level1button, 0.3)
level2 = Button.Button(232, 242, level2button, 0.3)
level3 = Button.Button(348, 358, level3button, 0.3)
level4 = Button.Button(464, 474, level4button, 0.3)
level5 = Button.Button(580, 590, level5button, 0.3)

pause = pygame.image.load("pause.png").convert_alpha()
pause_button = Button.Button(940, 1, pause, 0.2)

confirm = pygame.image.load("confirm.png").convert_alpha()
confirm_button = Button.Button(0, 95, confirm, 0.4)

recipe = pygame.image.load("recipebook.png").convert_alpha()
recipe_book = Button.Button(800, 1, recipe, 0.2)

Return = pygame.image.load("Return.png").convert_alpha()
return_button = Button.Button(0, 0, Return, 0.2)

### sounds ###

ding = pygame.mixer.Sound("service-bell-ring-14610.mp3")
ding.set_volume(0.4)
coffee = pygame.mixer.Sound("coffee_V1.mp3")
coffee.set_volume(0.4)
ambience = pygame.mixer.Sound("ambience.mp3")
```

```
ambience.set_volume(0.4)
sinks = pygame.mixer.Sound("sinks_V11.mp3")
sinks.set_volume(0.4)
cups = pygame.mixer.Sound("cups_V1.mp3")
cups.set_volume(0.4)
sizzle = pygame.mixer.Sound("sizzle.mp3")
sizzle.set_volume(0.4)
keyboard = pygame.mixer.Sound("keyboards_V1.mp3")
keyboard.set_volume(0.4)
chopping = pygame.mixer.Sound("chops_V1.mp3")
chopping.set_volume(0.4)
elevator = pygame.mixer.Sound("jazz music.mp3")
elevator.set_volume(0.4)

### texts and fonts ###

font = pygame.font.SysFont("Comic Sans MS", 90)
font1 = pygame.font.SysFont("Comic Sans MS", 45)
font2 = pygame.font.SysFont("Comic Sans MS", 30)
textsurface = font.render("Chai Central", True, (0,0,0))
textsurface2 = font.render("Levels", True, (0,0,0))

title = font.render("Settings", True, (0,0,0))
pausemenu = font.render("Game paused", True, (0,0,0))
pausemenu1 = font1.render("Press c to continue or q to quit", True, (0,0,0))

# cm = coffee machine, i = interactives, cb = chopping board, cr = cash register

cm1 = pygame.Rect(551, 449, 76, 146)
cm2 = pygame.Rect(892, 459, 107, 155)
pan = pygame.Rect(643, 514, 83, 37)
cb = pygame.Rect(383, 534, 129, 59)
cup = pygame.Rect(29, 517, 83, 64)
cr = pygame.Rect(152, 400, 180, 112)
sink = pygame.Rect(944, 628, 55, 48)

i = pygame.Surface((146,76))
i = pygame.Surface((155,107))
i = pygame.Surface((37,83))
i = pygame.Surface((59,129))
```

```
i = pygame.Surface((64,83))
i = pygame.Surface((112,180))
i = pygame.Surface((48,55))

### all procedures, classes etc###

# pause screen
def pauses():
    elevator.play()
    paused = True
    while paused:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
                sys.exit()
            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_c:
                    paused = False
                    elevator.stop()
                elif event.key == pygame.K_q:
                    pygame.quit()
                    sys.exit()
        screen.fill((146, 144, 106))
        screen.blit(pausemenu, (245,15))
        screen.blit(pausemenu1, (190,200))
        screen.blit(me1, (350,357))
        pygame.display.update()
        clock.tick(5)

def recipebook():
    paused = True
    while paused:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
                sys.exit()
        screen.fill((0,0,0))
        screen.blit(recip, (0,0))
        if return_button.draw(screen):
            paused = False
```

```
pygame.display.update()
clock.tick(5)

# initial start up for the main game
num = 0
def startup():
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
    screen.fill((255, 255, 255))
    screen.blit(screen2, (0,0))
    screen.blit(points, (0,0))
    point = font1.render(str(num), True, (0,0,0))
    screen.blit(point, (299,12))
    if pause_button.draw(screen2):
        pauses()

# the actual game requiring the user
correction = ""
def main_game():
    global correction
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
        if event.type == pygame.MOUSEBUTTONDOWN:
            if cm1.collidepoint(event.pos):
                print("pouring coffee")
                coffee.play()
                correction = correction+"cm1"
                print(correction)
            if cm2.collidepoint(event.pos):
                print("pouring coffee")
                coffee.play()
                correction = correction+"cm2"
                print(correction)
            if pan.collidepoint(event.pos):
                print("warming pan...")
```

```

        sizzle.play()
        correction = correction+"pan"
        print(correction)
    if cb.collidepoint(event.pos):
        print("chopping ingredients...")
        chopping.play()
        correction = correction+"cb"
        print(correction)
    if cup.collidepoint(event.pos):
        print("one cup taken")
        cups.play()
        correction = correction+"cup"
        print(correction)
    if cr.collidepoint(event.pos):
        print("cash register")
        keyboard.play()
        correction = correction+"cr"
        print(correction)
    if sink.collidepoint(event.pos):
        print("washing...")
        sinks.play()
        correction = correction+"sink"
        print(correction)
    pygame.display.update()

### customer class & sprites ###

customerlist=["sprite1.png", "sprite2.png"]

class Customer(pygame.sprite.Sprite):
    def __init__(self, image, x, y):
        pygame.sprite.Sprite.__init__(self)
        image = pygame.image.load(peoples)
        self.image = pygame.transform.scale(image, (image.get_width() / 2, image.get_height() / 2))
        self.rect = self.image.get_rect()
        self.rect.center = (x, y)

class timer():
    def __init__(self, x, y, w, h, max_time):
        self.x = x

        self.y = y
        self.w = w
        self.h = h
        self.time = max_time
        self.max_time = max_time
    def draw(self, surface):
        ratio = self.time / self.max_time
        rect1 = pygame.draw.rect(surface, "green", (self.x, self.y, self.w, self.h))
        rect2 = pygame.draw.rect(surface, "red", (self.x, self.y, self.w, self.h * ratio))

# randomly selects elements for the customer's "order"
def teastuff():
    assam = "cm2"+"pan"+"sink"+"cb"
    masala = "cr"+"cup"+"cm1"+"sink"
    badshah = "pan"+"cup"+"cm2"+"cm1"
    royal = "cr"+"cm2"+"cup"+"sink"
    tulsi = "cb"+"pan"+"sink"+"cm1"
    darjeeling = "cup"+"cr"+"pan"+"sink"
    kashmiri = "cm1"+"cm2"+"sink"+"cup"

    tea = [("assam", assam), ("masala", masala), ("badshah", badshah), ("royal", royal), ("tulsi", tulsi), ("darjeeling", darjeeling), ("kashmiri", kashmiri)]

    menu1 = random.choice(tea)
    menu2 = random.choice(tea)
    teal, tea2 = menu1
    i1, i2 = menu2
    return teal, tea2
    return i1, i2

peoples = random.choice(customerlist)
customers = pygame.sprite.Group()
customer = Customer(peoples, 445, 400)
customers.add(customer)

```

```
timing = 100
timervalue = 73
#timer value is the amount of time user gets to complete order
timer = timer(370, 200, 20, 100, timervalue)

def game():
    global t0
    global t1
    global dt
    global customers
    global tea1
    global tea2
    global i1
    global i2
    global t4
    global t2
    global dt2
    global correction
    global num
    global count
    t1= time.time()
    dt = t1 - t0
    startup()
    if dt >= 3:
        for event in pygame.event.get():
            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_r:
                    correction = ""
                    print("restarted recipe")
        customers.update()
        customers.draw(screen)
        screen.blit(speechbubble, (430,120))
        # tea1+" "+i1 represents the customer's order on screen
        speech = "I want"+" "+(tea1+" "+"and"+" "+i1)
```

```
customer_speech = font2.render(speech, True, (0,0,0))
screen.blit(customer_speech, (435,155))
t2 = time.time()
dt2 = t2 - t4
timer.time = dt2
timer.draw(screen)
if timer.time>=timervalue:
    t0 = t1
    t4 = t2
    count = count + 1
    peoples = random.choice(customerlist)
    customers = pygame.sprite.Group()
    customer = Customer(peoples, 445, 400)
    customers.add(customer)
    correction = ""
    tea1, tea2 = teastuff()
    i1, i2 = teastuff()
main_game()
#checks user input over answer
if confirm_button.draw(screen):
    # tea2+i2 represents the user's answer
    if tea2+i2 == correction:
        num = num + 1
        count = count +1
        t0 = t1
        t4 = t2
        print("correct")
        correction = ""
        tea1, tea2 = teastuff()
        i1, i2 = teastuff()
        peoples = random.choice(customerlist)
        customers = pygame.sprite.Group()
        customer = Customer(peoples, 445, 400)
        customers.add(customer)
        pygame.display.update()
```

```
        else:
            count = count +1
            t0 = t1
            t4 = t2
            print("incorrect")
            correction = ""
            teal, tea2 = teastuff()
            i1, i2 = teastuff()
            peoples = random.choice(customerlist)
            customers = pygame.sprite.Group()
            customer = Customer(peoples, 445, 400)
            customers.add(customer)
            pygame.display.update()

### main game loop ###

game_state = 1
t0 = time.time()
t4 = time.time()
teal, tea2 = teastuff()
i1, i2 = teastuff()
run = True
while run:

    #creates a timer
    t1 = time.time()
    t2 = time.time()

    #menu screen
    if game_state==1:
```

```
    screen.fill((146, 144, 106))
    screen.blit(textsurface, (245,15))
    if level_button.draw(screen):
        game_state=2
    if tutorial_button.draw(screen):
        game_state=3
    if settings_button.draw(screen):
        game_state=4
    if quitgame_button.draw(screen):
        event.type==pygame.QUIT
        run = False
    pygame.display.update()

#levels screen
elif game_state==2:
    screen.fill((146, 144, 106))
    screen.blit(textsurface2, (375,15))
    if level1.draw(screen):
        game_state=5
    if level2.draw(screen):
        game_state=6
    if level3.draw(screen):
        game_state=7
    if level4.draw(screen):
        game_state=8
    if level5.draw(screen):
        game_state=9
    pygame.display.update()

#tutorial level
elif game_state==3:
    num = 0
    screen.fill((255, 255, 255))
    screen.blit(screen2, (0,0))
    startup()
```

```
        main_game()
        customers.update()
        customers.draw(screen)
        pygame.display.update()

#settings screen
elif game_state==4:
    screen.fill((146, 144, 106))
    screen.blit(title, (310,15))
    screen.blit(settingpic, (0,0))
    pygame.display.update()

#main game after level is chosen

#level 1
elif game_state==5:
    num = 0
    t0 = t1
    t4 = t2
    count = 0
    while game_state == 5:
        game()
        if recipe_book.draw(screen):
            recipebook()
        if count == 3:
            game_state = 10
            break
        pygame.display.update()

#level 2
elif game_state == 6:
    num = 0
    t0 = t1
    t4 = t2
    count = 0
```

```
    while game_state == 6:
        game()
        if recipe_book.draw(screen):
            recipebook()
        if count == 3:
            game_state = 10
            break
        pygame.display.update()

#level 3
elif game_state == 7:
    num = 0
    t0 = t1
    t4 = t2
    count = 0
    while game_state == 7:
        game()
        if recipe_book.draw(screen):
            recipebook()
        if count == 3:
            game_state = 10
            break
        pygame.display.update()

#level 4
elif game_state == 8:
    num = 0
    t0 = t1
    t4 = t2
    count = 0
    while game_state == 8:
        game()
        if count == 5:
            game_state = 10
            break
```

```
pygame.display.update()

#level 5
elif game_state == 9:
    num = 0
    t0 = t1
    t4 = t2
    count = 0
    while game_state == 9:
        game()
        if count == 5:
            game_state = 10
            break
        pygame.display.update()

#endgame screen
elif game_state == 10:
    screen.fill((0,0,0))
    screen.blit(endgame, (0,0))
    cubes = font2.render(str(num)+"/"+str(count)+" sugar cubes!", True, (0,0,0))
    screen.blit(cubes, (584, 138))
    pygame.display.update()
    for event in pygame.event.get():
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_e:
                game_state = 2
        if event.type==pygame.QUIT:
            run = False

#the main event loop
for event in pygame.event.get():
    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_ESCAPE:
            game_state=1
    if event.type==pygame.QUIT:

        run = False
    #makes the rect of input transparent on screen.
    i.set_alpha(128)
    i.fill((255,255,255))
    screen2.blit(i, (0,0))
    pygame.display.update()

pygame.display.update()
clock.tick(60)
pygame.quit()
```

DISCLAIMER

- All images and sounds found are copyright free.
- Any articles and videos used to create this game have been linked in the corresponding section with the name of the creator.